



AKTU

B.Tech III-Sem / MCA



CS IT & CS Allied

COA Computer Organization & Architecture

Crash Course



Unit-3 in One Shot



By Pragya Ma'am

Unit-III : Control Unit**AKTU : Syllabus**

Control Unit: Instruction types, formats, instruction cycles and sub cycles (fetch and execute etc), micro operations, execution of a complete instruction. Program Control, Reduced Instruction Set Computer, Pipelining. Hardwire and micro programmed control: micro programme sequencing, concept of horizontal and vertical microprogramming.

AKTU QUESTIONS

Q.1	What is RISC and how it differs with CISC?
Q.2	What are differences between hardwired and microprogrammed control unit?
Q.3	Differentiate between Horizontal & Vertical microprogramming
Q.4	Explain the pipelining in detail with neat and clean diagram and explain different type of pipeline
Q.5	Explain the following term. i)Micro operation ii) Microinstruction iii) Microprogram iv) Micro Code. vi) pipelining and Parallelism vii) linear and non-linear pipeline
Q.6	Explain different types of micro operations.
Q.7	Explain Microinstruction format and microprogrammed control unit
Q.8	Explain Microprogram sequencer
Q9	What is instruction format. Explain different types of instruction format

AKTU QUESTIONS(Numericals)

Q.1	A vertical microprogrammed control unit support 512 instructions. The system is using 8 conditional flag and contain 31 control, signals. Each instruction on an average has 1 micro operations. Calculate the approximate size of memory in bytes
Q.2	Evaluate the arithmetic statement $X = (A+B)*(C+D)$ using general register computer with Zero Address ,One Address, Two Address and Three Address instruction formats.
Q.3	A non-pipelined system takes 50 ns to process a task. The same task can be processed in a six-segment pipeline with a clock cycle of 10 ns. Determine the speedup ratio of the pipeline for 100 tasks. What is the maximum speedup that can be achieved?

INSTRUCTION CYCLE

- The **instruction cycle**, also known as the **fetch-decode-execute cycle**, is a **fundamental concept** that describes how a **computer processes a machine language instruction**.

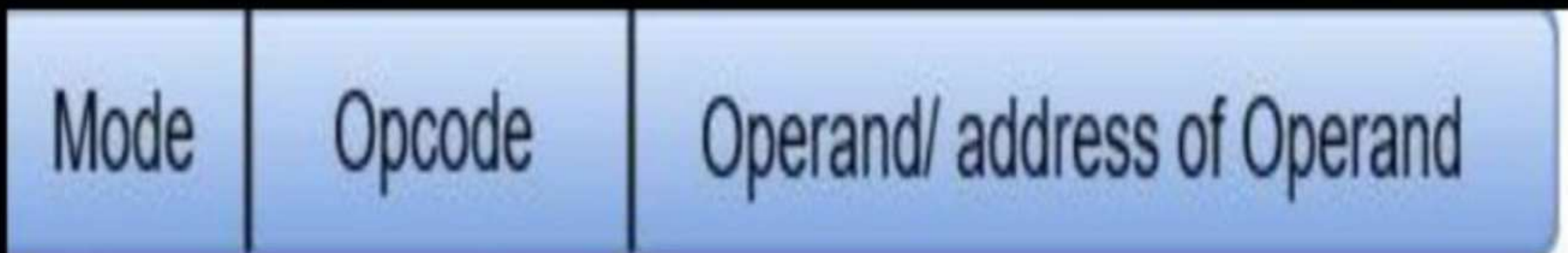
➤ The major steps in the instruction cycle are.-

➤ **Fetch:** In the fetch cycle, the **CPU retrieves the instruction from memory**. The instruction is typically stored at the **address specified by the program counter (PC)**. The PC is then incremented to point to the next instruction in memory.

➤ **Decode:** In the decode cycle, the **CPU interprets the instruction and determines what operation needs to be performed**. This involves identifying the **opcode and any operands that are needed to execute the instruction**.

➤ **Execute:** In the execute cycle, the **CPU performs the operation specified by the instruction**. This may involve **reading or writing data from or to memory**, performing **arithmetic or logic operations on data**, or manipulating the **control flow of the program**.

- Computer instructions are a set of machine language instructions that a particular processor understands and executes..
- An instruction comprises of groups called fields. These fields include:
- The Operation code (Opcode) field which specifies the operation to be performed.
- The Address field which contains the location of the operand, i.e., register or memory location.
- The Mode field which specifies how the operand will be located.



INSTRUCTION

- A basic computer has three instruction code formats which are:
- Memory - reference instruction
- Register - reference instruction
- Input-Output instruction

Memory - reference instruction

I=0 FOR Direct memory reference

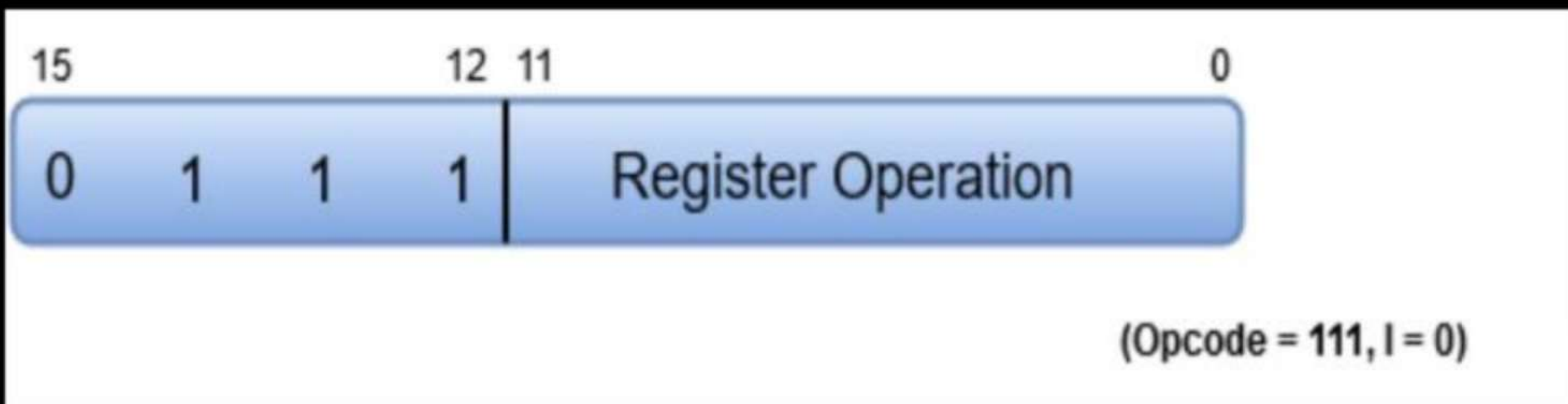
I=1 for indirect memory reference



(Opcode = 000 through 110)

Register - reference instruction

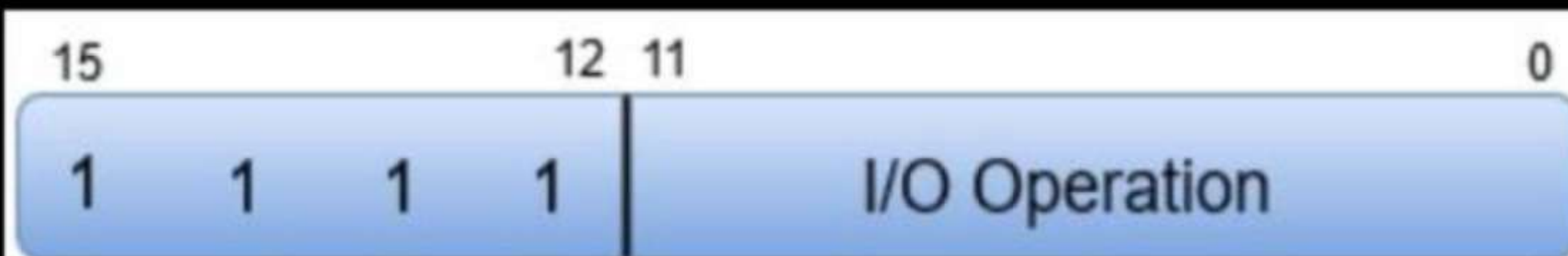
- The Register-reference instructions are represented by the Opcode 111 with a 0 in the leftmost bit (bit 15) of the instruction. A Register-reference instruction specifies an operation on or a test of the AC (Accumulator) register.



INSTRUCTION

➤ Input-Output instruction

Just like the Register-reference instruction, an Input-Output instruction does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction. The remaining 12 bits are used to specify the type of the input-output operation or test performed



(Opcode = 111, I = 1)

MICRO OPERATIONS

- Micro operations are **basic, elementary operations** that a **computer's central processing unit (CPU)** performs on **data stored in its registers.**

Types Of Micro Operations:

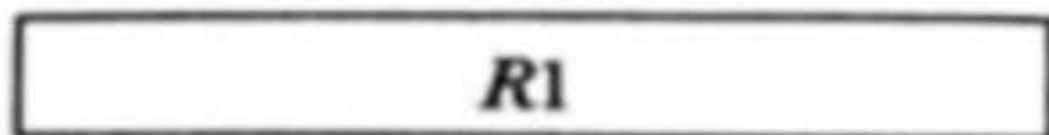
- **Arithmetic Micro Operations:** These operations involve mathematical calculations such as **addition, subtraction, multiplication, and division.**
- **Logical Micro Operations:** Logical micro operations include bitwise operations like **AND, OR, NOT, and XOR**, which are used for comparing and manipulating binary data.
- **Shift Micro Operations:** Shift operations involve moving bits within a **register left or right**, effectively multiplying or dividing by powers of two.
- **Transfer Micro Operations:** Transfer operations move data between **different registers or memory locations within the CPU**

Arithmetic Micro-operation

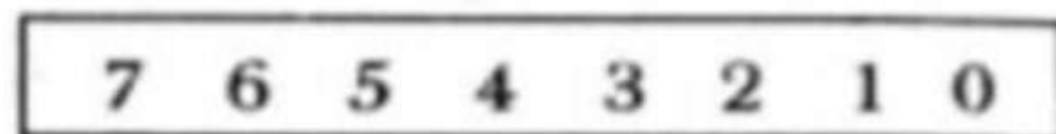
Symbolic Representation	Description
$R3 \leftarrow R1 + R2$ (ADDITION)	The contents of R1 plus R2 are transferred to R3.
$R3 \leftarrow R1 - R2$ (SUBTRACTION)	The contents of R1 minus R2 are transferred to R3.
$R2 \leftarrow R2'$	Complement the contents of R2(1's complement).
$R2 \leftarrow R2' + 1$	2's complement the contents of R2(negate).
$R3 \leftarrow R1 + R2' + 1$	R1 plus the 2's complement of R2(subtraction).
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by one.
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by one.

➤ The symbolic notation used to describe the micro-operation transfer among registers is called RTL (Register Transfer Language).

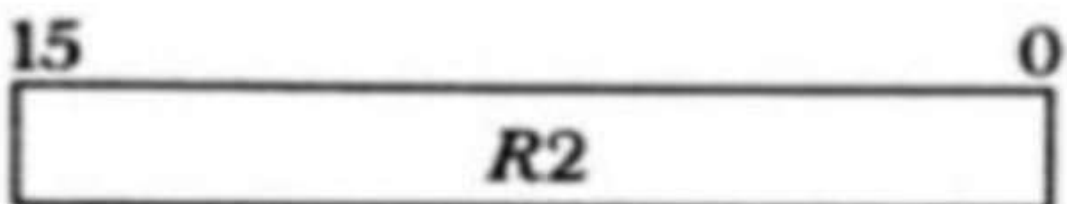
Figure 4-1 Block diagram of register.



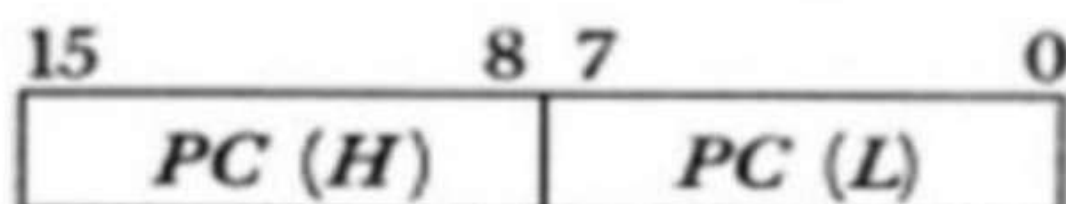
(a) Register *R*



(b) Showing individual bits



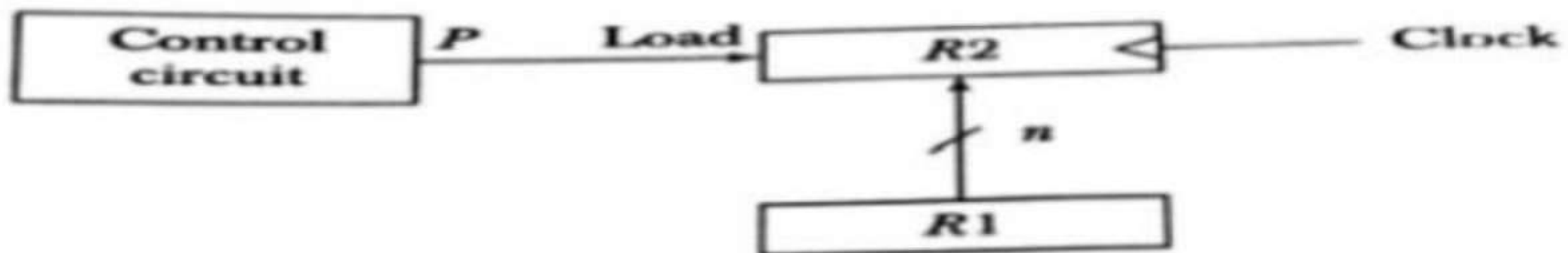
(c) Numbering of bits



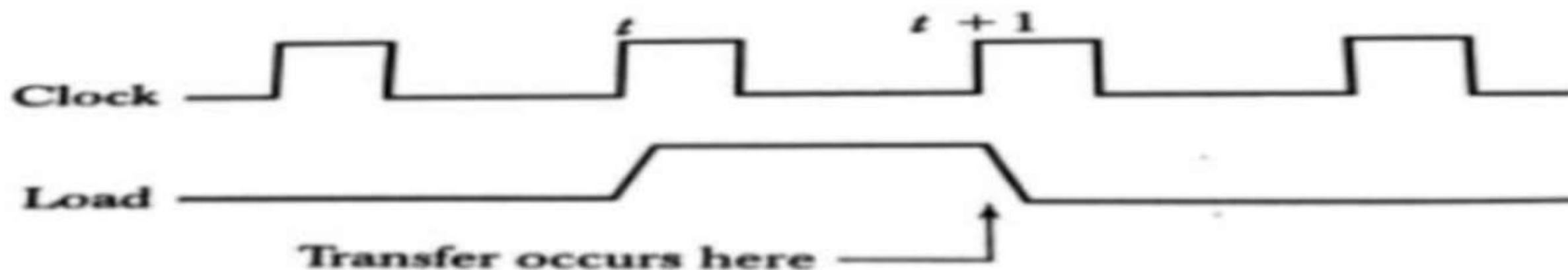
(d) Divided into two parts

Register transfer micro-operation

Figure 4-2 Transfer from $R1$ to $R2$ when $p = 1$.



(a) Block diagram



(b) Timing diagram

Register transfer micro-operation

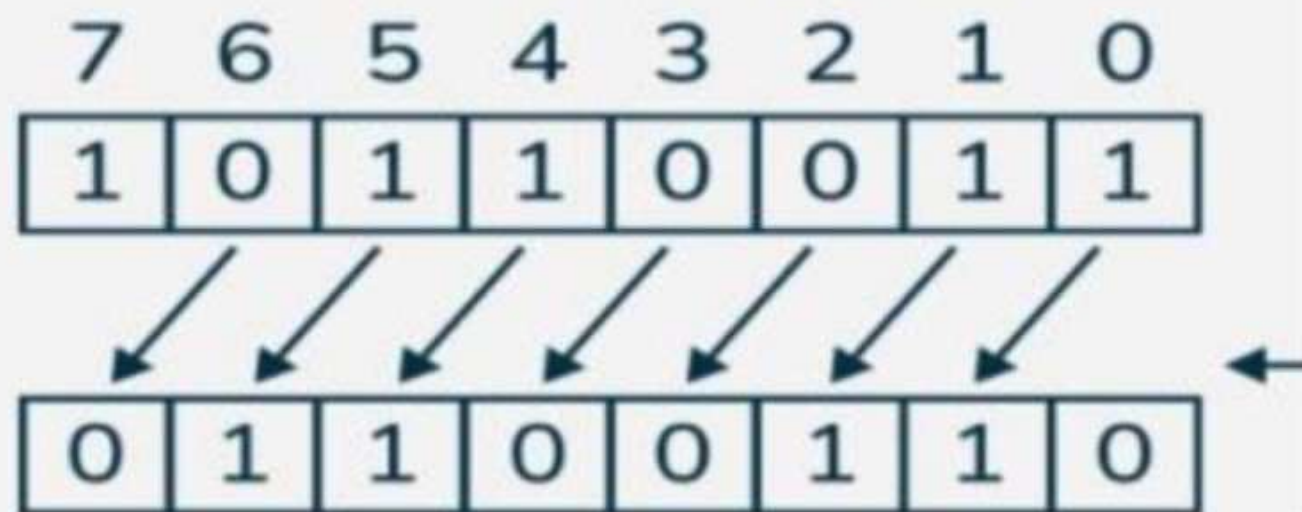
Symbol	Description	Example
Letters(and numerals)	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow <--	Denotes transfer of information	R2 <-- R1
Comma ,	Separates two micro operations	R2 <-- R1, R1 <-- R2

Shift micro operations

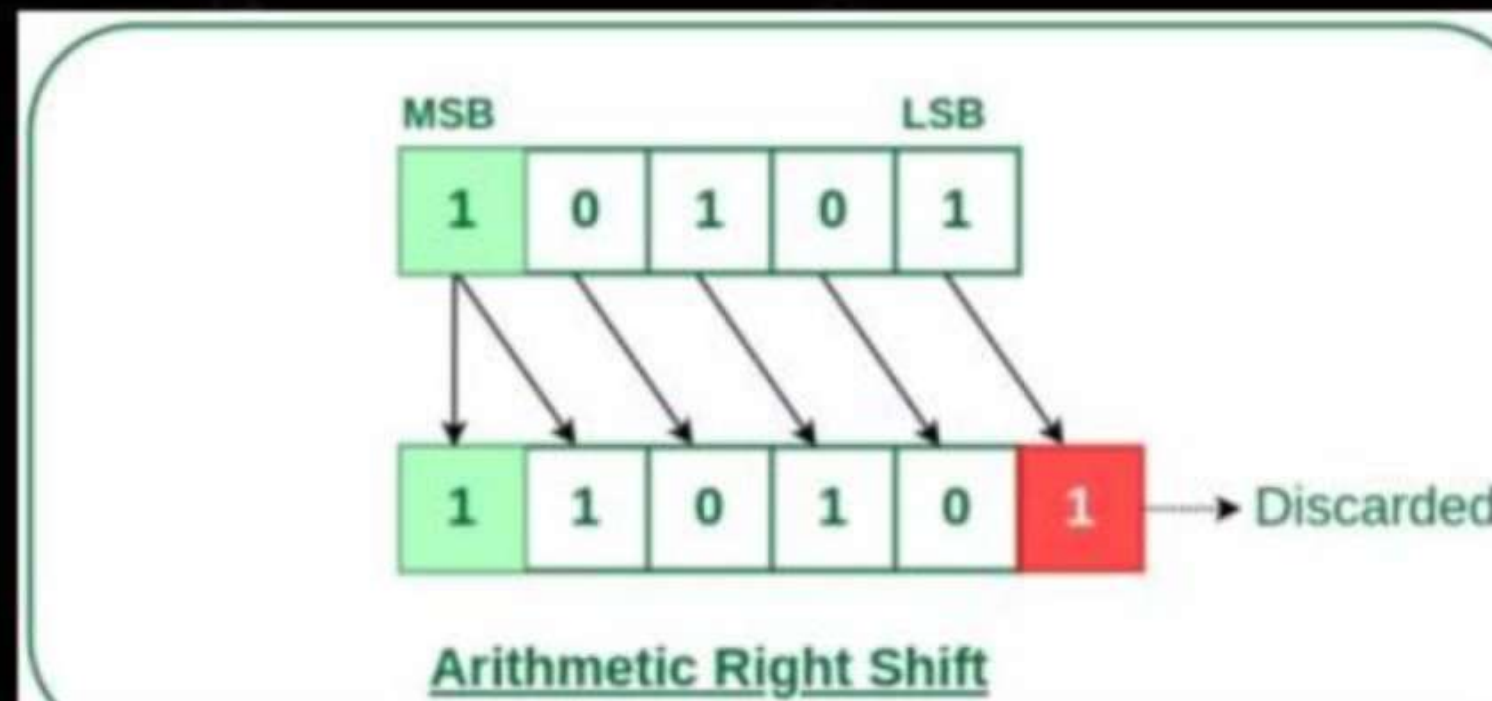
1. Arithmetic shift

a. Arithmetic shift left

Shift Left Arithmetic



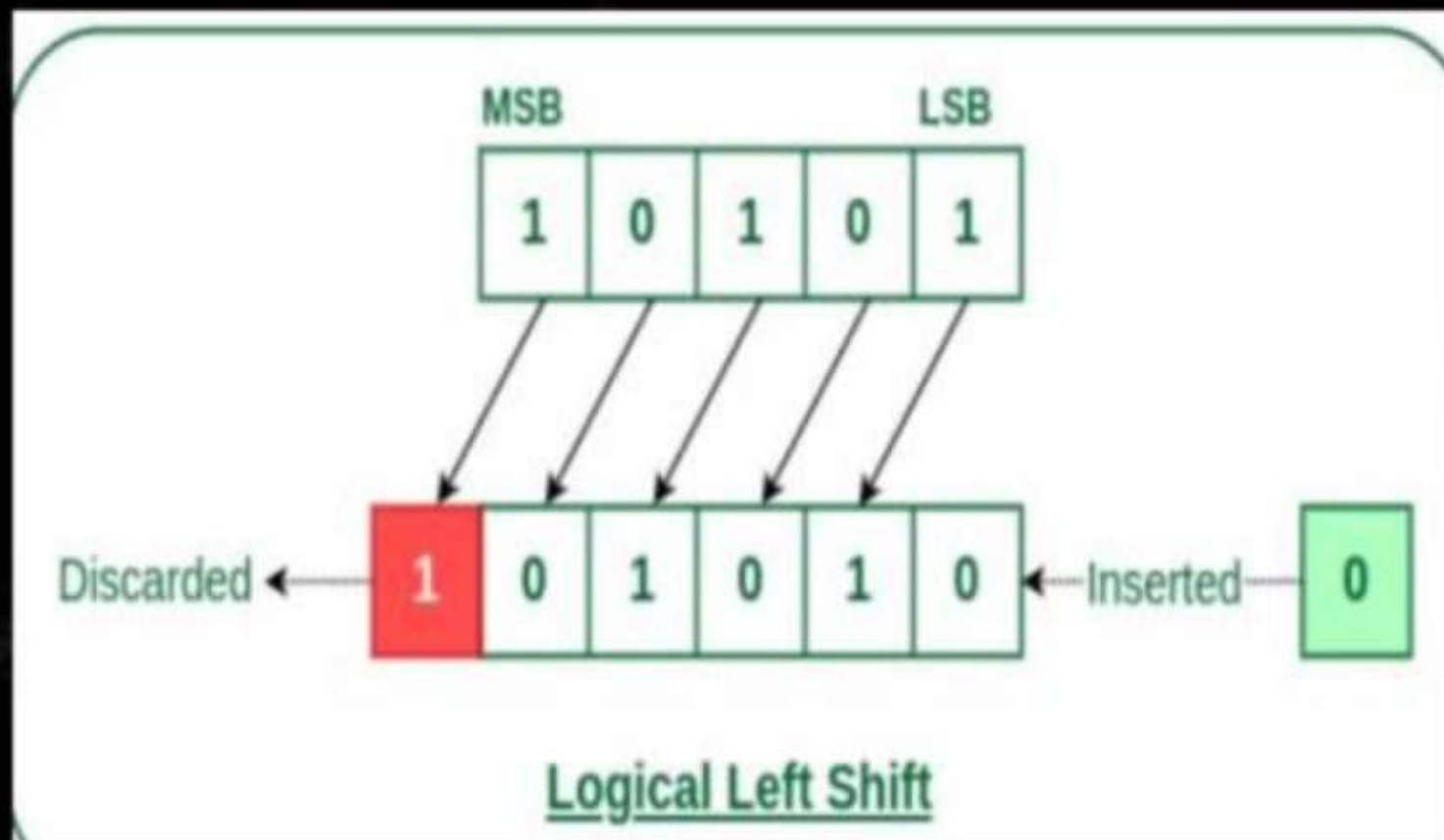
b. Arithmetic shift right



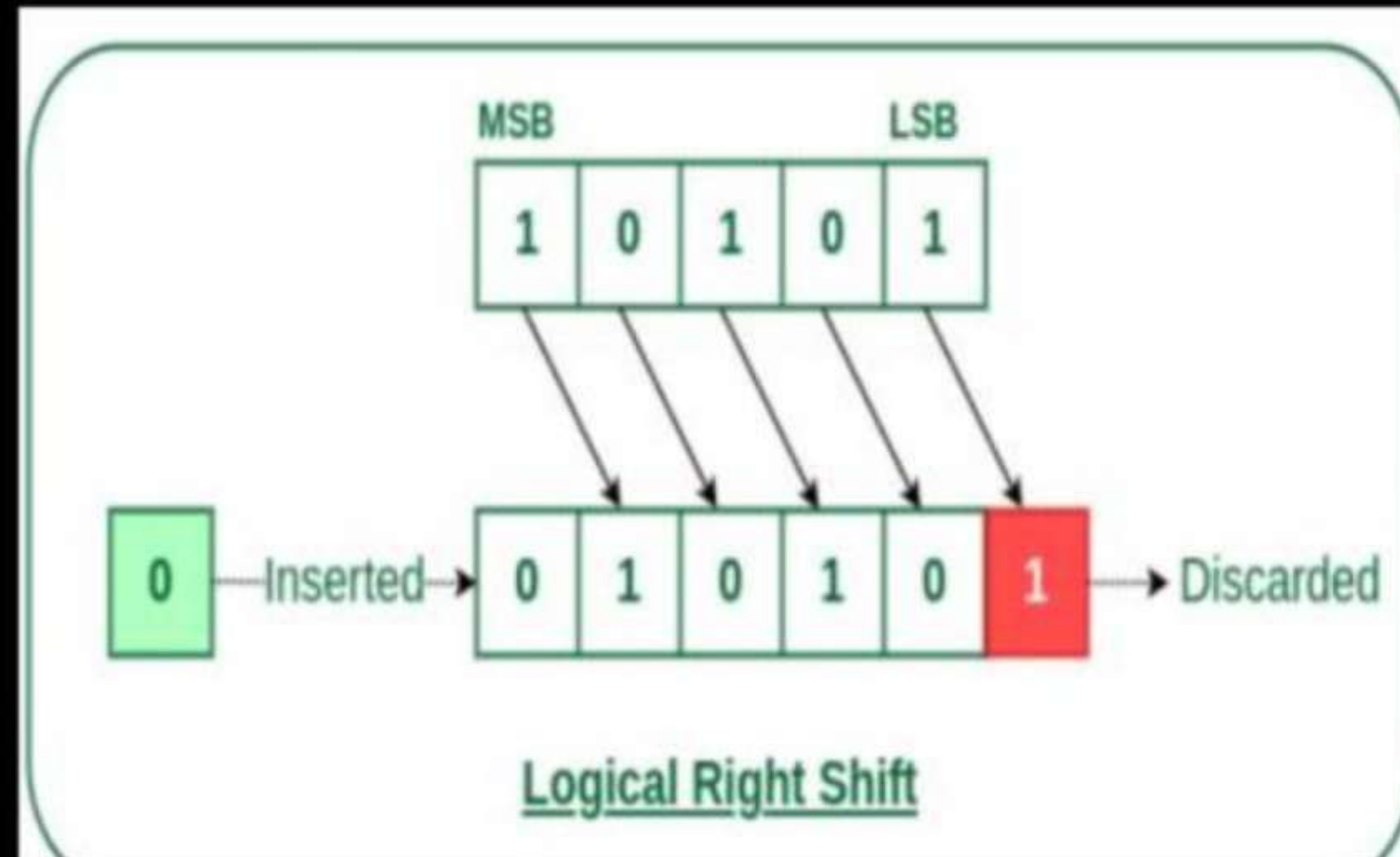
Shift micro operations

1. Logical shift

a. Logical shift left



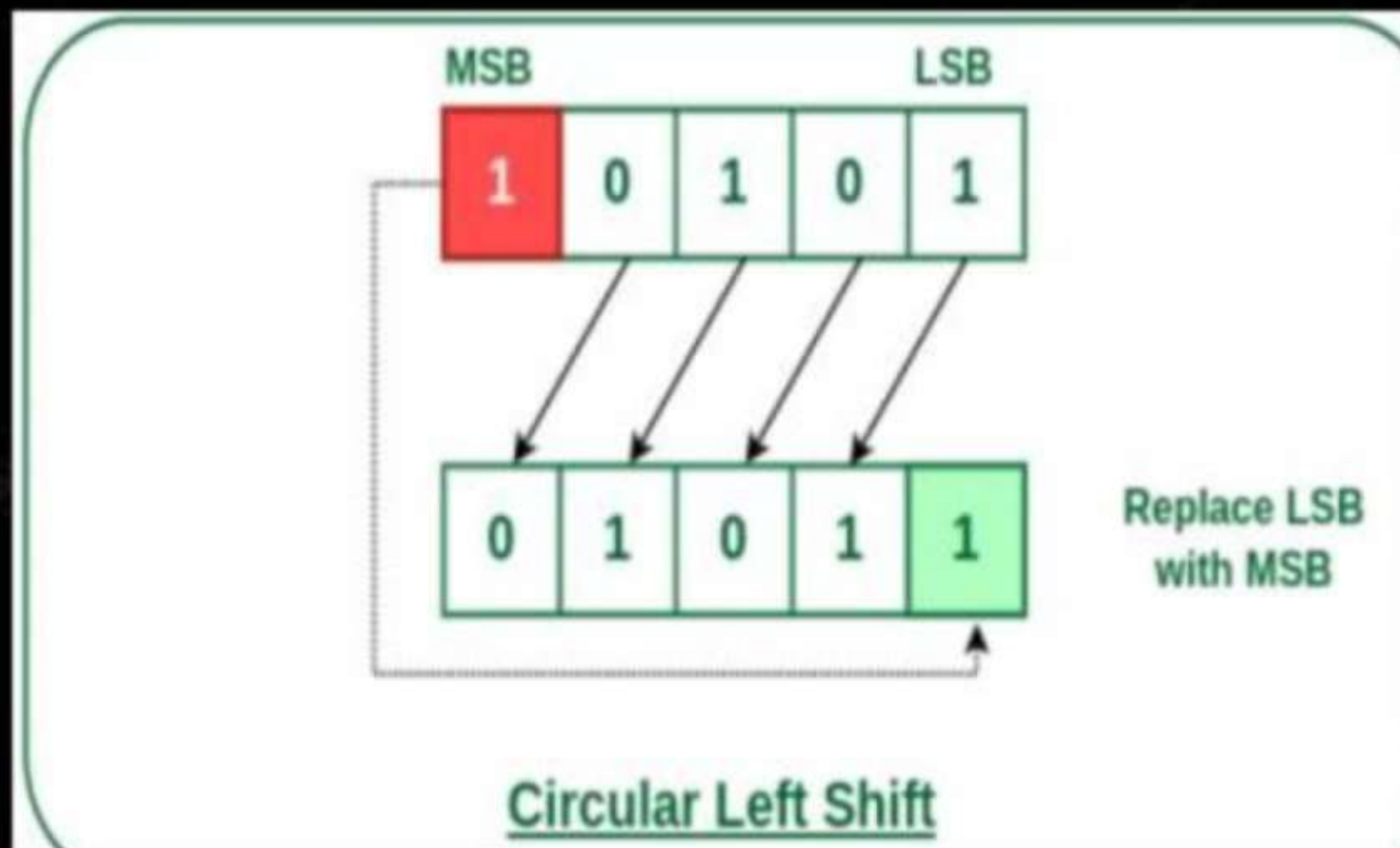
b. Logical shift right



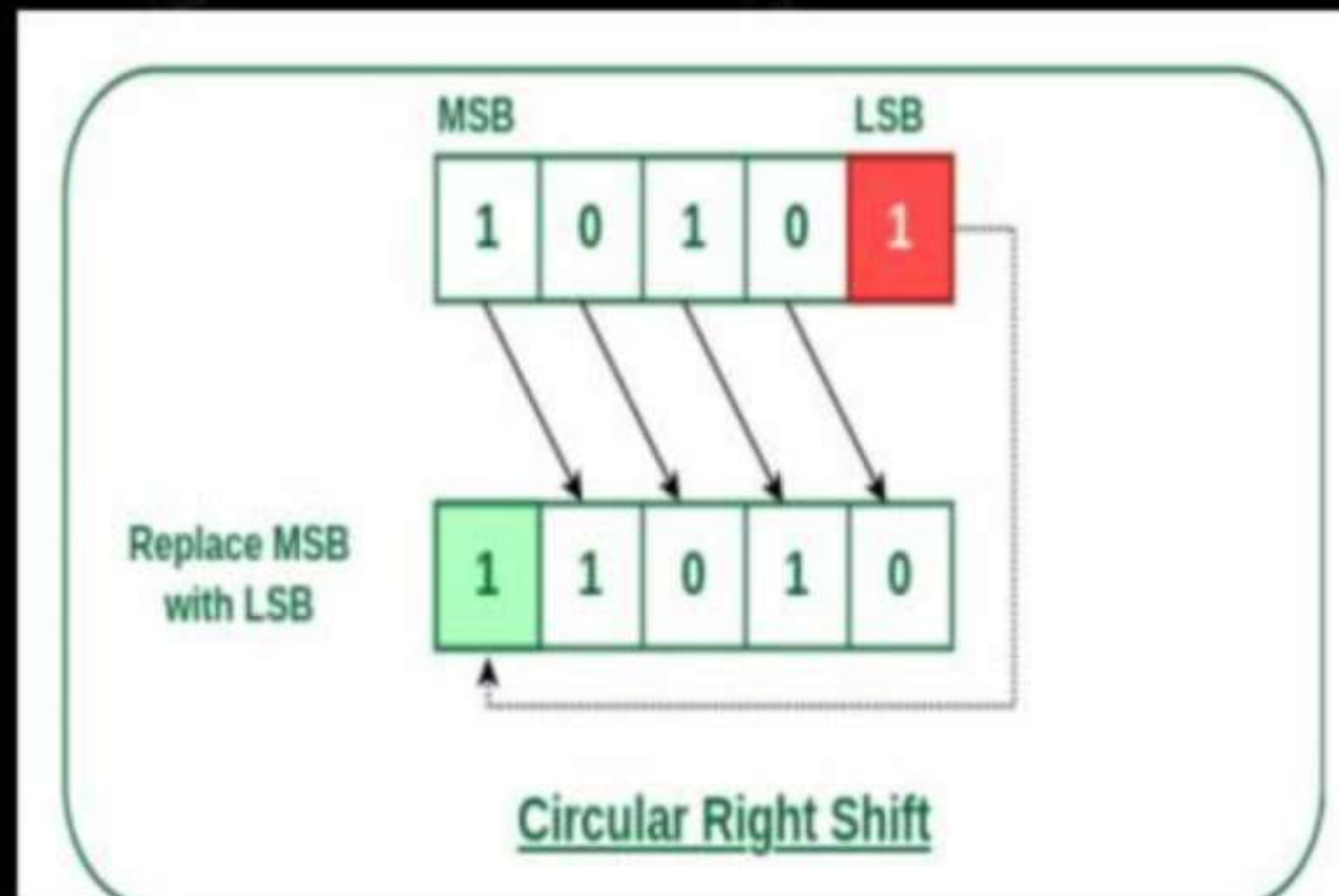
Shift micro operations

1. circular shift

a. circular shift left



b. circular shift right



Logical Micro operations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = x.y$	$F \leftarrow A \wedge B$	AND
$F_2 = x.y'$	$F \leftarrow A \wedge \bar{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'.y$	$F \leftarrow \bar{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y'$	$F \leftarrow A \oplus B$	Exclusive OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive NOR
$F_{10} = y'$	$F \leftarrow \bar{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \bar{B}$	
$F_{12} = x'$	$F \leftarrow \bar{A}$	Complement A

$F_{13} = x' + y$	$F \leftarrow \bar{A} \vee B$	
$F_{14} = (x.y)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

DIFFERENCE BETWEEN-

RISC(Reduced Instruction Set Computer)	CISC(Complex Instruction Set Computer Processor)
Code size is large	Code size is small
An instruction executed in a single clock cycle	Instruction takes more than one clock cycle
An instruction fit in one word .(1WORD =2 BYTE)	Instructions are larger than the size of one word
Fixed sized instructions	Variable sized instructions
Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
Requires more number of registers	Requires less number of registers

DIFFERENCE BETWEEN-

RISC	CISC
Simple and limited addressing modes.	Complex and more addressing modes.
RISC is Reduced Instruction Cycle.	CISC is Complex Instruction Cycle.
It consumes the low power.	It consumes more/high power.
RISC is highly pipelined.	CISC is less pipelined.
RISC required more <u>RAM</u> .	CISC required less RAM.
RISC reduces the cycles per instruction at the cost of the number of instructions per program	CISC tries to minimize the number of instructions per program but at the cost of increasing the number of cycles per instruction

➤ To **improve** the performance of a CPU we **have two options**:

- 1) Improve the **hardware** by introducing **faster circuits**.
- 2) Arrange the **hardware** such that more than **one operation can be performed at the same time**.

Since there is a limit on **the speed of hardware** and the **cost of faster circuits is quite high**, we have to adopt the 2nd option

Pipelining

➤ It is a **process of arrangement of hardware elements** of the CPU such that **its overall performance is increased**. Simultaneous execution of **more than one instruction takes place** in a pipelined processor.

➤ Consider a water bottle packaging plant. Let there be 3 stages that a bottle should pass through, Inserting the bottle(I), Filling water in the bottle(F), and Sealing the bottle(S).

Without pipelining = $9/3$ minutes = 3m

```
I F S | | | | |
| | | I F S | | |
| | | | | | I F S (9 minutes)
```

With pipelining = $5/3$ minutes = 1.67m

```
I F S | |
| I F S |
| | I F S (5 minutes)
```

PIPELINING(PHASE TIMING DIAGRAM)

Non-overlapped execution:

Stage / Cycle	1	2	3	4	5	6	7	8
S1	I ₁				I ₂			
S2		I ₁				I ₂		
S3			I ₁				I ₂	
S4				I ₁				I ₂

Total time = 8 Cycle

Overlapped execution:

Stage / Cycle	1	2	3	4	5
S1	I ₁	I ₂			
S2		I ₁	I ₂		
S3			I ₁	I ₂	
S4				I ₁	I ₂

5 cycle

5 stages PIPELINING

RISC processor has 5 stage instruction pipeline to execute all the instructions in the RISC instruction set. Following are the 5 stages of the RISC pipeline with their respective operations:

Stage 1 (Instruction Fetch) In this stage the CPU reads instructions from the address in the memory whose value is present in the **program counter**.

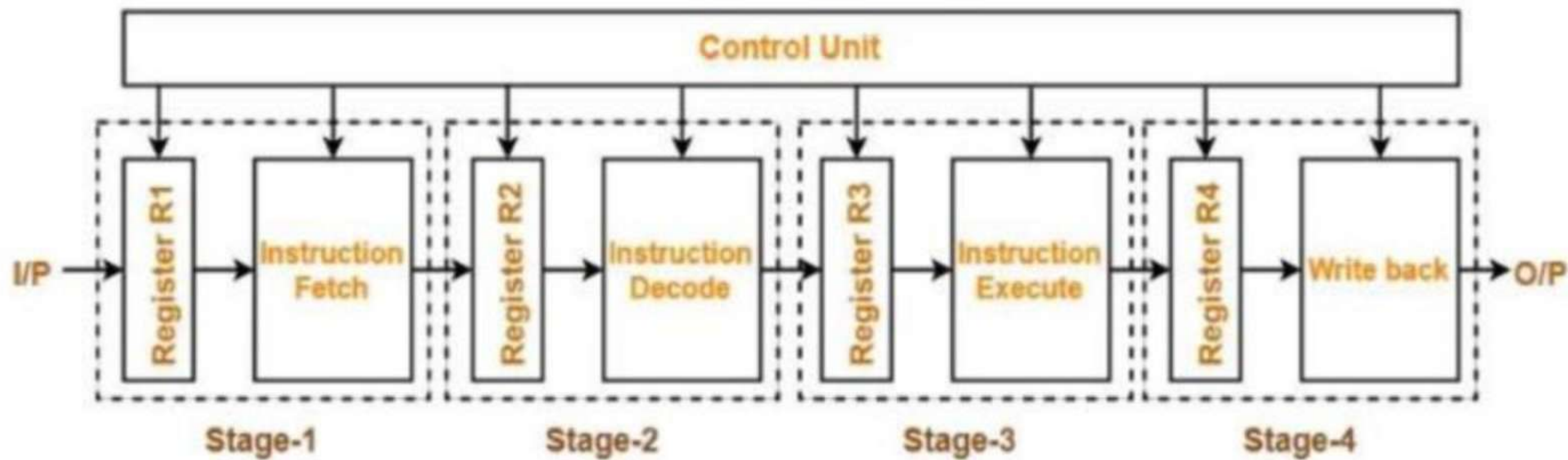
Stage 2 (Instruction Decode) In this stage, instruction is decoded and the register file is accessed to get the values from the registers used in the instruction.

Stage 3 (Instruction Execute) In this stage, ALU operations are performed.

Stage 4 (Memory Access) In this stage, memory operands are read and written from/to the memory that is present in the instruction.

Stage 5 (Write Back) In this stage, computed/fetched value is written back to the register present in the instructions.

4 STAGE PIPELINING



Four Stage Instruction Pipeline

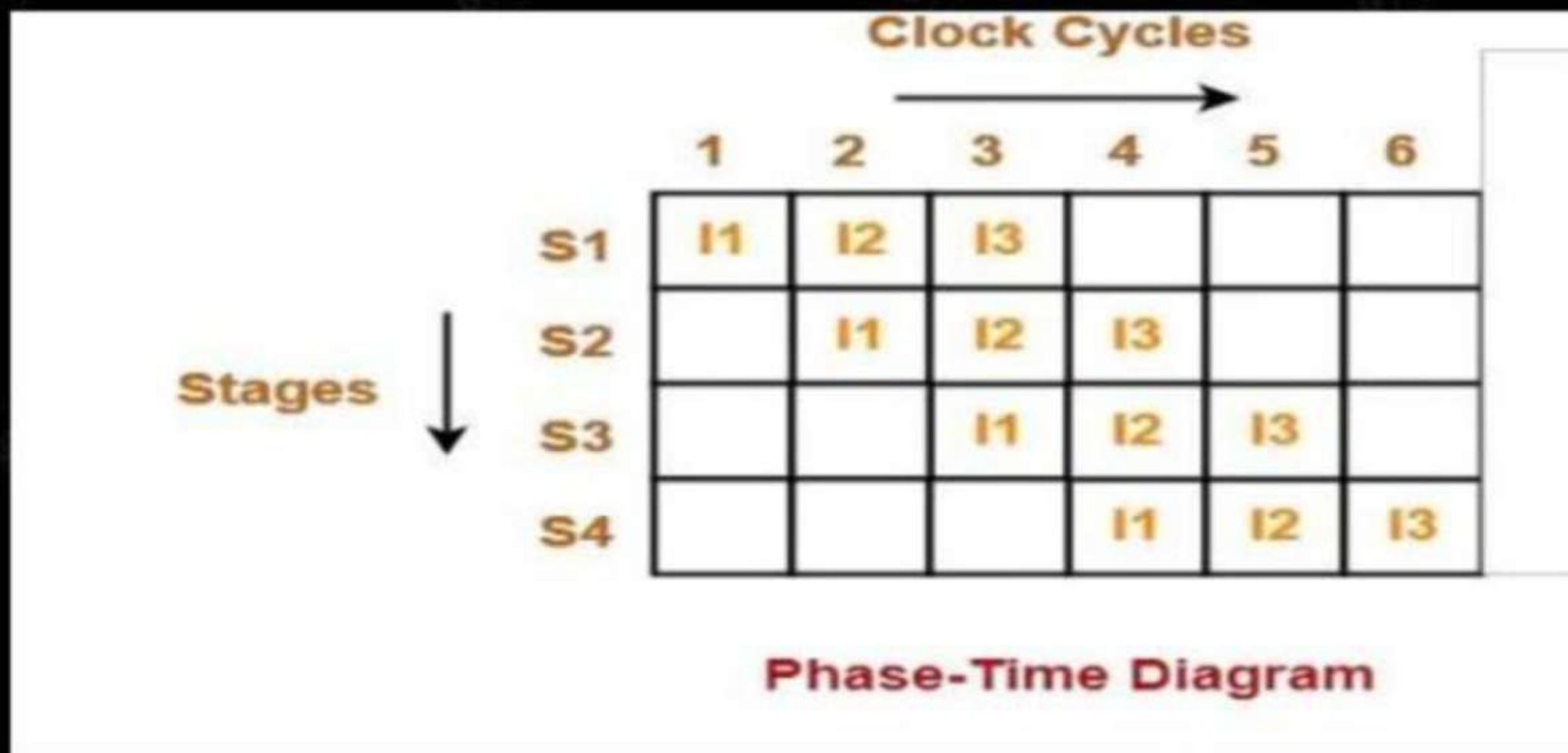
4 STAGE PIPELINING

Stage-01: At stage-01, First functional unit performs instruction fetch. It fetches the instruction to be executed.

Stage-02: At stage-02, Second functional unit performs instruction decode. It decodes the instruction to be executed.

Stage-03: At stage-03, Third functional unit performs instruction execution. It executes the instruction.

Stage-04: stage-04, Fourth functional unit performs write back. It writes back the result so obtained after executing the instruction.



Performance of a pipelined processor Consider a ' k ' segment pipeline with clock cycle time as ' T_p '.

Let there be ' n ' tasks to be completed in the pipelined processor. Now, the first instruction is going to take ' k ' cycles to come out of the pipeline but the other ' $n - 1$ ' instructions will take only ' 1 ' cycle each, i.e, a total of ' $n - 1$ ' cycles. So, time taken to execute ' n ' instructions in a pipelined processor:

$$\begin{aligned} ET_{\text{pipeline}} &= k + n - 1 \text{ cycles} \\ &= (k + n - 1) T_p \end{aligned}$$

In the same case, for a non-pipelined processor, the execution time of ' n ' instructions will be:

$$ET_{\text{non-pipeline}} = n * k * T_p$$

So, **speedup (S)** of the **pipelined processor** over the **non-pipelined processor**, when '**n**' tasks are executed on **the same processor** is:

$S = \text{Performance of non-pipelined processor} / \text{Performance of pipelined processor}$

As the performance of a processor is inversely proportional to the execution time, we have,

$$S = ET_{\text{non-pipeline}} / ET_{\text{pipeline}}$$

$$\Rightarrow S = [n * k * T_p] / [(k + n - 1) * T_p]$$

$$S = [n * k] / [k + n - 1]$$

When the number of tasks '**n**' is significantly larger than **k**, that is, $n \gg k$

$$S = n * k / n$$

$$S = k$$

where 'k' are the number of stages in the pipeline. Also, Efficiency = Given speed up / Max speed up = S / S_{max} We know that $S_{max} = k$ So,

$$\text{Efficiency} = S / k$$

Throughput = Number of instructions / Total time to complete the instructions So,

$$\text{Throughput} = n / (k + n - 1) * T_p$$

Note: The cycles per instruction (CPI) value of an ideal pipelined processor is 1

Performance of pipeline is measured using two main metrics as Throughput and latency.

Throughput:

- It measures **number of instructions completed per unit time**.
- It represents **overall processing speed** of pipeline.
- **Higher throughput** indicates **processing speed** of pipeline.
- Calculated as, **throughput = number of instructions executed / execution time**.
- It can be affected by **pipeline length, clock frequency, efficiency** of instruction execution and presence of pipeline hazards or stalls.

Latency:

- It measure **time taken for a single instruction to complete its execution.**
- It represents **delay or time it takes for an instruction to pass through pipeline stages.**
- Lower latency **indicates better performance .**
- It is calculated as, **Latency= Execution time/ Number of instruction executed.**
- It is influenced by **pipeline length, depth, clock cycle time, instruction dependencies and pipeline hazards..**

Linear Pipeline

In linear pipeline a **series of processors** are **connected together** in a **serial manner**.

Linear pipeline is also called as **static pipeline** as it **performs fixed functions**.

The output is always **produced** from the **last block**.

Linear pipeline has **linear connections**.

It generates a **single reservation table**.

It allows **easy functional partitioning**.

Non-Linear Pipeline

In Non-Linear pipeline **different processor** are present at **different stages**.

Non-Linear pipelines is also called as **dynamic pipeline** as it performs **different functions**.

The output is **not necessarily produced** from the **last block**.

Non-Linear pipeline **has feedback** and **feed-forward connections**.

It can generate **more than one reservation table**.

Functional partitioning is **difficult** in **non-linear pipeline**.

 Aspect	Pipelining	 Parallelism
Nature of Concurrency	Achieves concurrency by breaking down the execution of a single instruction into stages and processing multiple instructions in different stages simultaneously.	Achieves concurrency by executing multiple instructions or tasks simultaneously, either at the instruction level, thread level, or data level.
Resource Usage	Uses a single set of hardware components that are shared among different stages of the pipeline.	Involves multiple sets of hardware components that can operate concurrently on different instructions or tasks.
Workflow	Each stage of the pipeline is responsible for a specific task, and as soon as one stage completes its task, it passes the result to the next stage.	Simultaneous execution of multiple instructions or tasks, either at the instruction level, thread level, or data level.

➤ Arithmetic Pipelining

➤ Arithmetic Pipelines are commonly used in various high-performance computers. They are used in order to implement floating-point operations, fixed-point multiplication, and other similar kinds of calculations that come up in scientific situations.

➤ Let's look at an example

➤ The inputs in the floating-point adder pipeline refer to two different normalized floating-point binary numbers. These are defined as follows:

➤ $A = X * 2^x = 0.9504 * 10^3$

➤ $B = Y * 2^y = 0.8200 * 10^2$

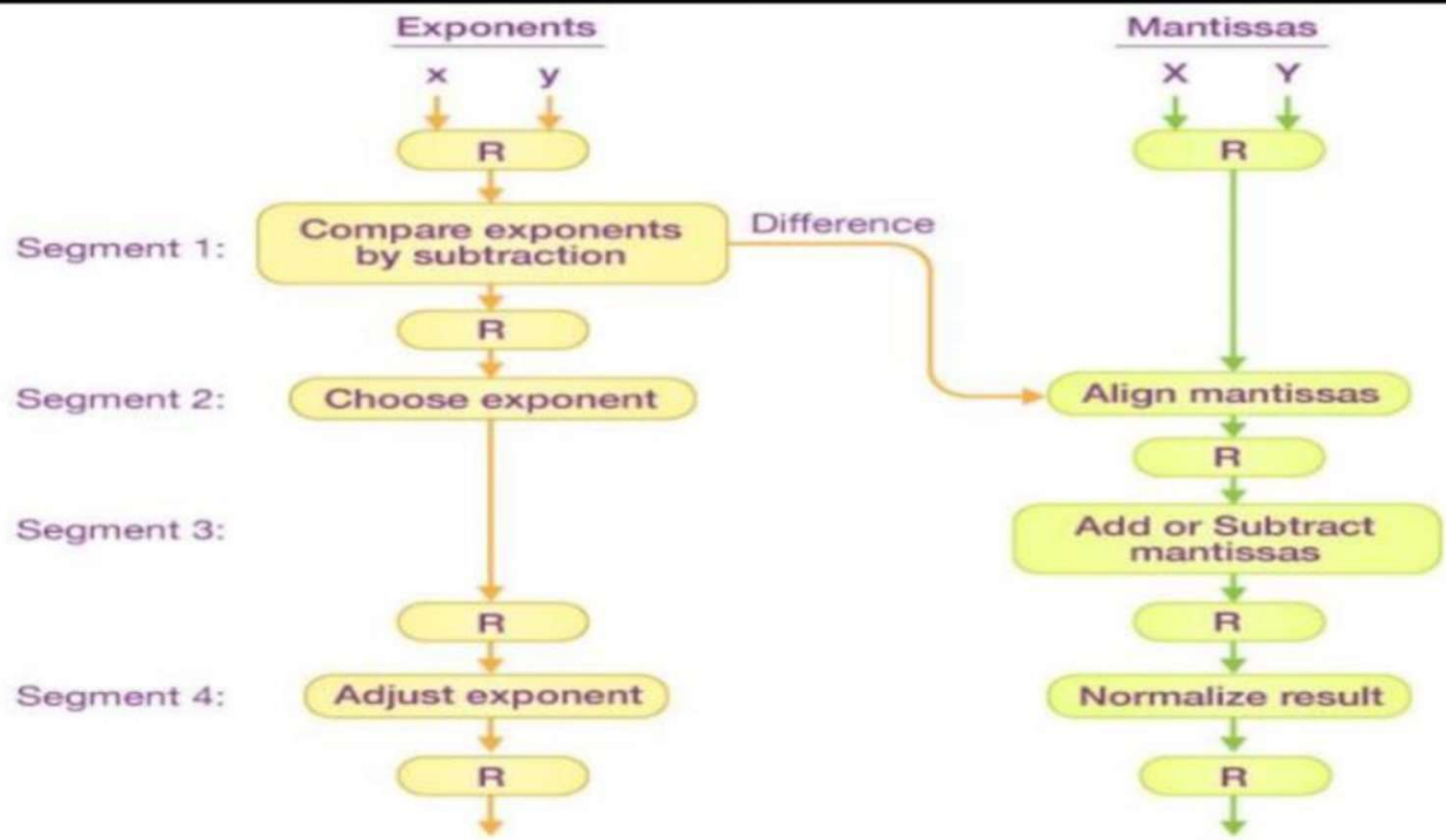
➤ Arithmetic Pipelining

Where x and y refer to the exponents and X and Y refer to two fractions representing the mantissa.

The floating-point addition and subtraction process is broken into four pieces.

1. Comparing the exponents using subtraction
2. Aligning the mantissa
3. Adding or subtracting the mantissa
4. Normalizing the result

Classification of pipelining



➤ Arithmetic Pipelining

Note: The registers are placed after every sub-operation in order to store the intermediate results.

1. Comparing Exponents by Subtraction

The difference between the exponents is calculated by subtracting them. The result's exponent is chosen to be the larger exponent.

The exponent difference, $3 - 2 = 1$, defines the total number of times the mantissa associated with the lesser exponent should be shifted to the right.

2. Aligning the Mantissa

As per the difference of exponents calculated in segment one, the mantissa corresponding with the smaller exponent would be moved.

$$A = 0.9504 * 10^3 \quad B = 0.08200 * 10^3$$

3. Adding the Mantissa

Both the mantissa would be added in the third segment.

$$C = A + B = 1.0324 * 10^3$$

4. Normalizing the Result

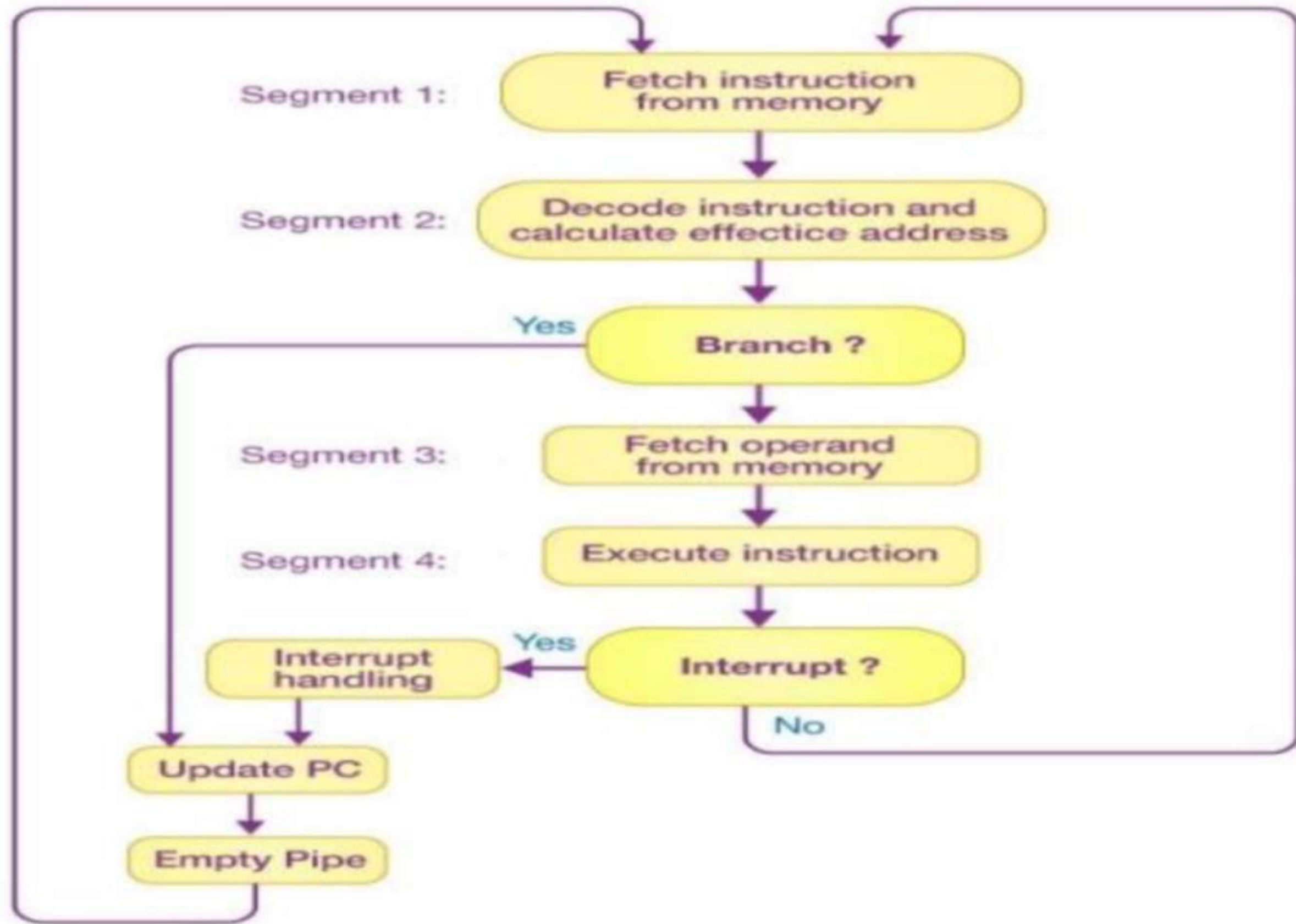
After the process of normalization, the result would be written as follows:

$$C = 0.1324 * 10^4$$

Instruction pipelining

Pipeline **processing** can happen not only in the **data stream** but also in the **instruction stream**. To perform tasks such as **fetching, decoding and execution of instructions**, In general, each and every instruction must be processed by the computer in the following order:

1. Fetching the instruction from memory
2. Decoding the obtained instruction
3. Calculating the effective address
4. Fetching the operands from the given memory
5. Execution of the instruction
6. Storing the result in a proper place



Classification of pipelining

instruction pipelining

four-segment instruction pipeline is illustrated in the block diagram given above. The instructional cycle is divided into four parts:

Segment 1

The implementation of the **instruction fetch segment** can be done using the **FIFO or first-in, first-out buffer**.

Segment 2

In the second segment, the **memory instruction is decoded**, and the **effective address is then determined** in a **separate arithmetic circuit**.

Segment 3

In the third segment, **some operands would be fetched from memory**.

Segment 4

The instructions would finally be executed in the very **last segment of a pipeline organisation**

	Stage	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction	1	FI	DA	FO	EX									
Branch	2		FI	DA	FO	EX								
	3			FI	DA	FO	EX							

➤ Control Unit :

The unit which directs the operation of the processor & is a part of the CPU is known as Control Unit.

It generates control signals for the operations of a computer

Types of Control Unit :

There are two types of control units as follows.

➤ Hardwired control unit

➤ Micro-programmed control unit

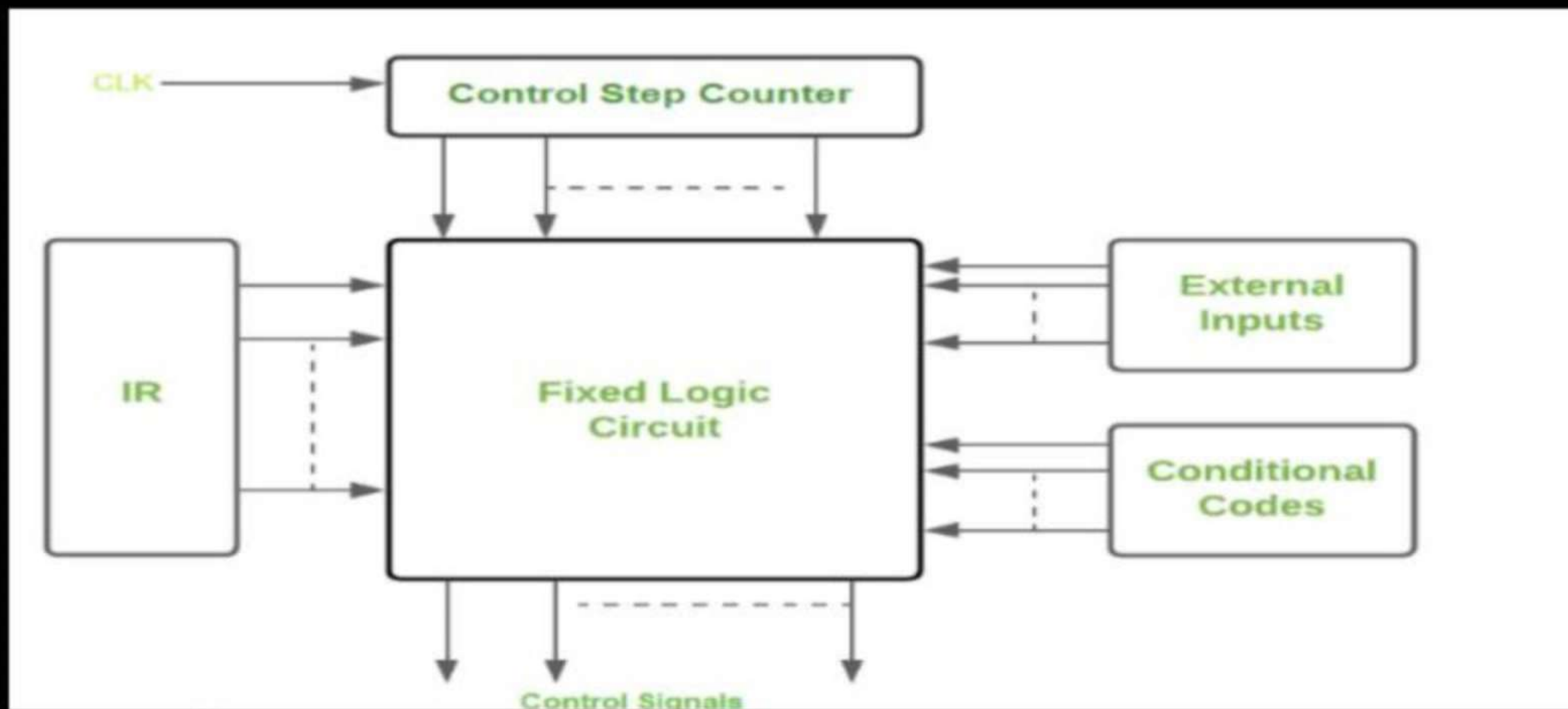
➤ The fundamental difference between hardwired and microprogrammed control unit is the **hardwired circuitry approach** where as the microprogram control unit is implemented by **programming**

Hardwired control unit

- These control units are designed using **hardware components** such as **sequential logic circuits** or **some finite state machines**, for example, **logic gates, flip-flops, decoders, resistors** and other digital circuits to generate a specific sequence of control signals.
- To **interpret the instructions & generate control signals**, the control unit **uses fixed logic circuits**.

- To **generate signals**, the **fixed logic circuits** use the **contents** of the **control step counter, Instruction Register (IR) & code flag**, and some **external input signals** such as interrupt signals.
- The fixed logic circuit is a **combinational circuit** made from **decoders & encoders**.
- The decoder **decodes** the instruction loaded **in IR** (Instruction Register) & **generates the signal** that serves as an **input to the encoder**.
- Also, **external input & conditional codes** act as an **input** to the **encoder**. The encoder **then accordingly** generates the **control signals based on the inputs**.

Hardwired control unit



ADVANTAGES

- Use of **combinational circuits** to generate signals, Hardwired Control Unit **is fast**.
- It depends on **number of gates**, how much **delay** can occur in **generation of control signals**.
- It can be **optimized** to produce the fast mode of operation.
- **Faster** than **micro-programmed** control unit.
- It **does not** require control memory.

DISADVANTAGES

- The **complexity of the design** increases as **we require** more **control signals** to be **generated** (need of more encoders & decoders)
- Modifications in the **control signals** are very difficult because it requires **rearranging of wires in the hardware circuit**.
- Adding a **new feature** is difficult & complex.
- It is **Expensive**.

Method to design hardwired control

1 state table method

2 Delay element

method

3 sequential counter

method

4 PLA Method

1. STATE TABLE METHOD

- The behavior of **control unit** is represented in the form of a **table**, which is known as the **state table**.
- Here, **each row** represents the **T-states** and the **columns** represent the **instructions**.
- Every **intersection** of the specific column to each row indicates which **control signal will be produced** in the corresponding **T- state of an instruction**.
- The **hardware circuitry** is designed for **each column**(i.e. for a specific instruction) for producing **control signals** in **different T-states**.

STATE TABLE METHOD

T - States	Instructions			
	I_1	I_2	...	I_N
T_1	$C_{1,1}$	$C_{1,2}$	...	$C_{1,N}$
T_2	$C_{2,1}$	$C_{2,2}$	...	$C_{2,N}$
...	...	...	...	...
T_M	$C_{M,1}$	$C_{M,2}$	...	$C_{M,N}$

$C_{1,1}$ means control signal to be produced in T-state(T_1) of Instruction (I_1)

Advantage –

- It is the **simplest method**.
- This method is mainly used for **small instruction set processors**

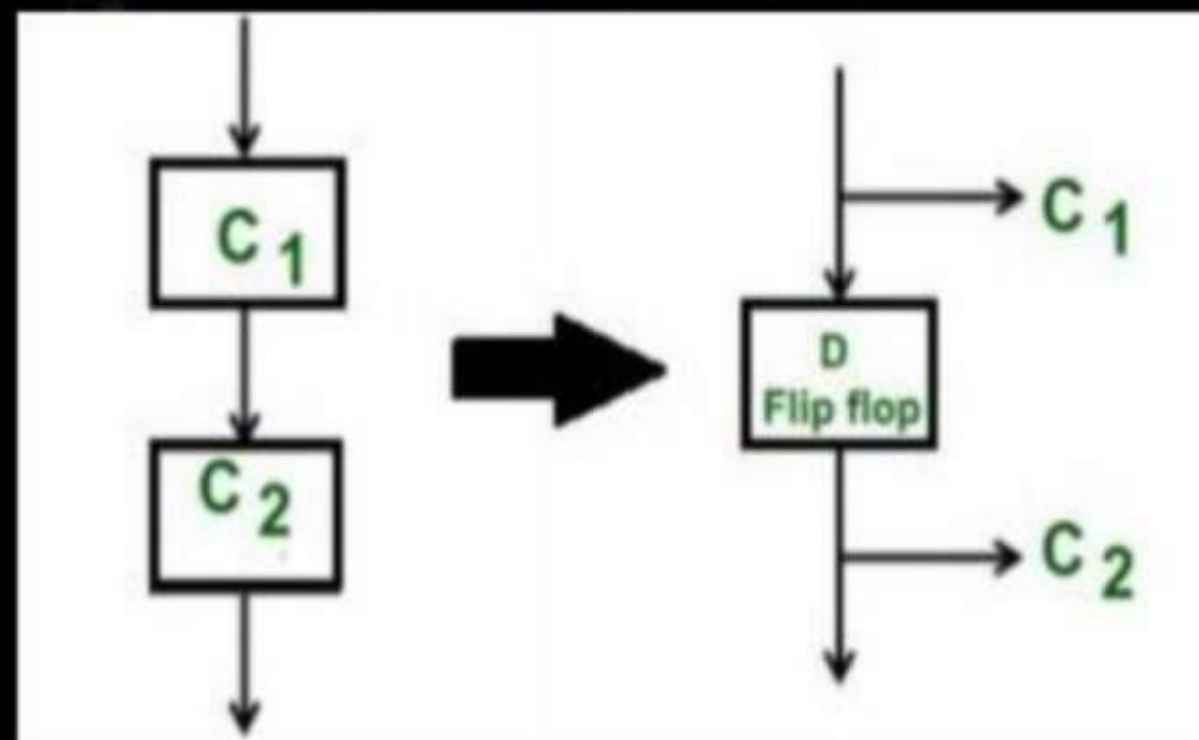
Disadvantages

- In **modern processors**, there is a **very large number of instruction set**. Therefore, the circuit becomes **complicated to design, difficult to debug**, and if we make **any modifications to the state table** then the large parts of **the circuit need to be changed**.

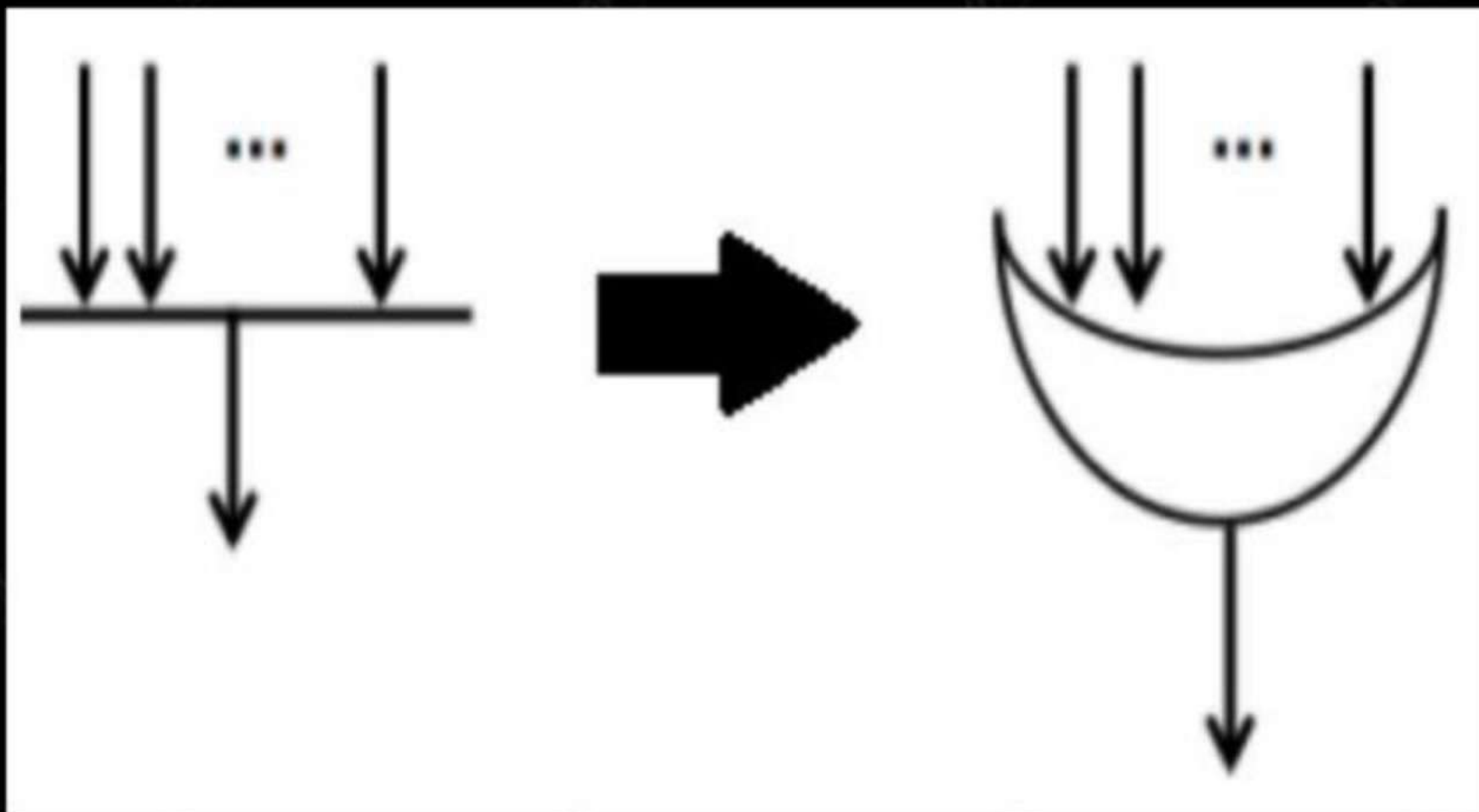
DELAY ELEMENT METHOD

- Here the **control unit behavior** is represented in the form of a **flowchart**.
- Each step in the **flowchart** represents a **control signal** that **needs to be produced** for processing the **instructions**.
- If **all the steps of the instructions** are **performed**, this means the **instruction is executed completely**.
- **Control signals** perform **micro-operations** and **each micro-operation** requires one **T-state**.

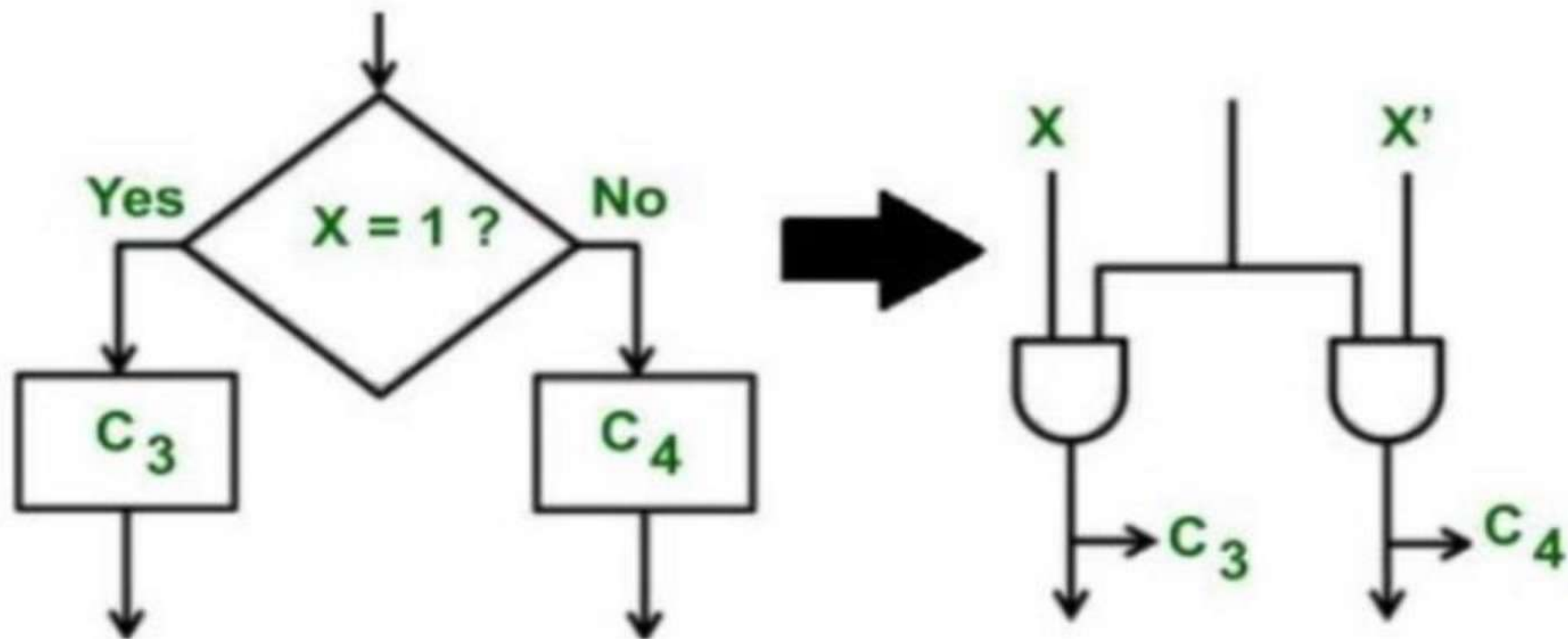
- For the **micro-operations** which are **independent**, they are required to be performed in **different T-state**. Therefore, **for every consecutive control signal** an **exactly 1-state delay** is required, which can be produced with the help of **D FF**.
- Therefore, **D Flip-Flops** are inserted between every **two consecutive control signals**



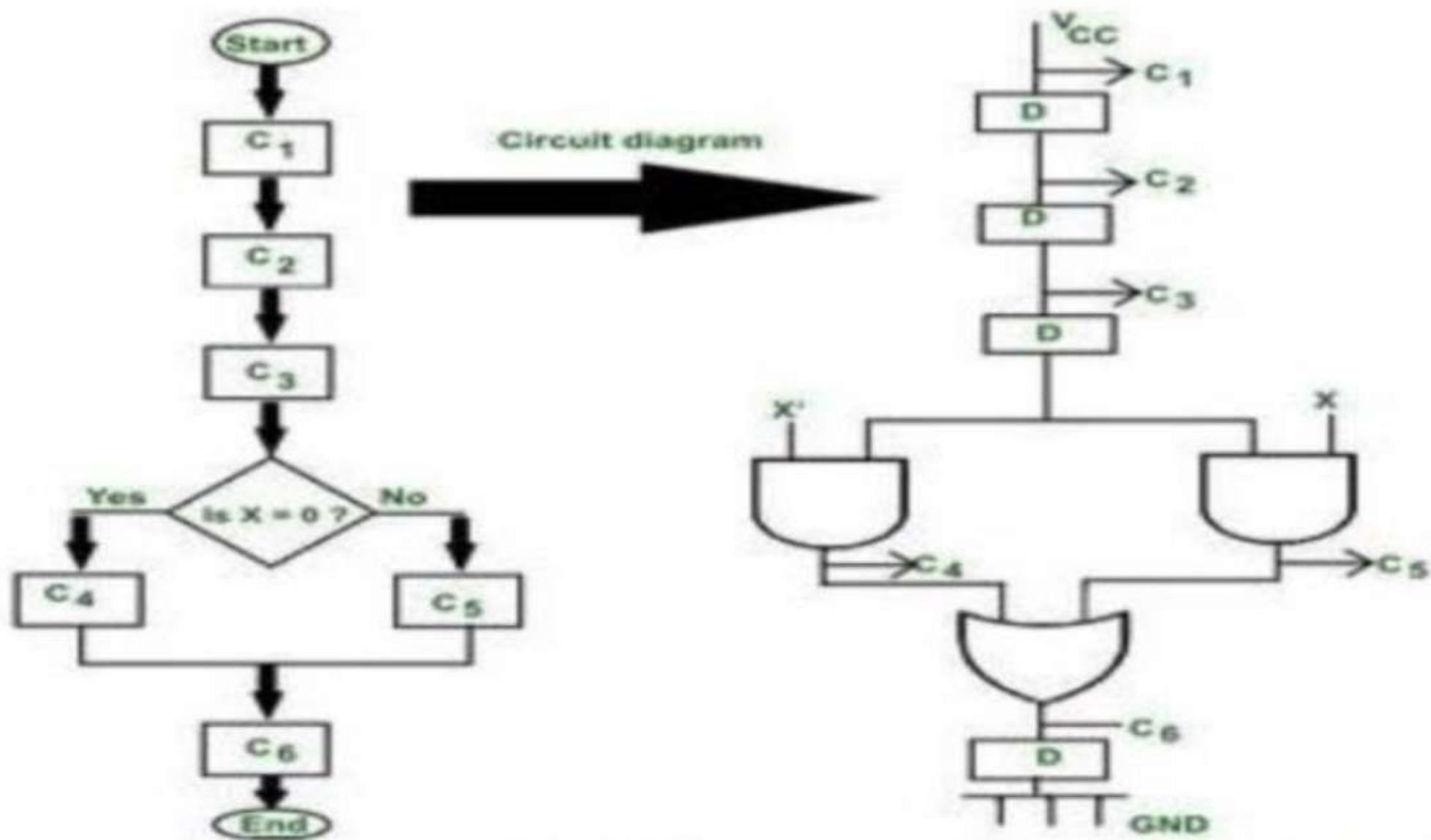
Hardwired control unit



Hardwired control unit



Hardwired control unit



Delay element method for generating control signals.

Hardwired control unit

Advantage –

- This method has a logical approach ,therefore it helps to reduce the circuit complexity.
- For the common control signals which need to be generated in every instruction, for them only one circuitry can be designed .

➤ Drawback –

- As the number of instructions increases , the number of D FF for generating delay is increased, so overall circuit complexity and cost increases.

Microprogrammed Control Unit

- It is a type of **control unit** where the **binary control variables** (control signals) are stored in a **special memory** called **control memory** as **microinstructions**. These microinstructions **dictate** how the **processor** **executes instructions**.

1. Control Memory

- It is a **special memory** in a **microprogrammed control unit** used **to store the microprogram** (a sequence of microinstructions).
- Each **microinstruction** generates **control signals** to execute **machine instructions**.

2. control word

It is a **binary string** (sequence of 1s and 0s) that represents the **control variables** at a **given time in a microprogrammed** control unit. **Each bit in the control word** activates or deactivates **specific control signals** needed for executing an **instruction**.

3. control variable

is a binary signal that enables or disables a specific micro-operation in a processor

Microinstruction

➤ A microinstruction is a **binary instruction** that **specifies one or more micro-operations** to be **executed in a single clock cycle**. It is stored in **control memory** and **generates control signals** for **CPU operations**.

- Example: A microinstruction may activate ALU addition, enable a register, and set memory read.

Microprogram

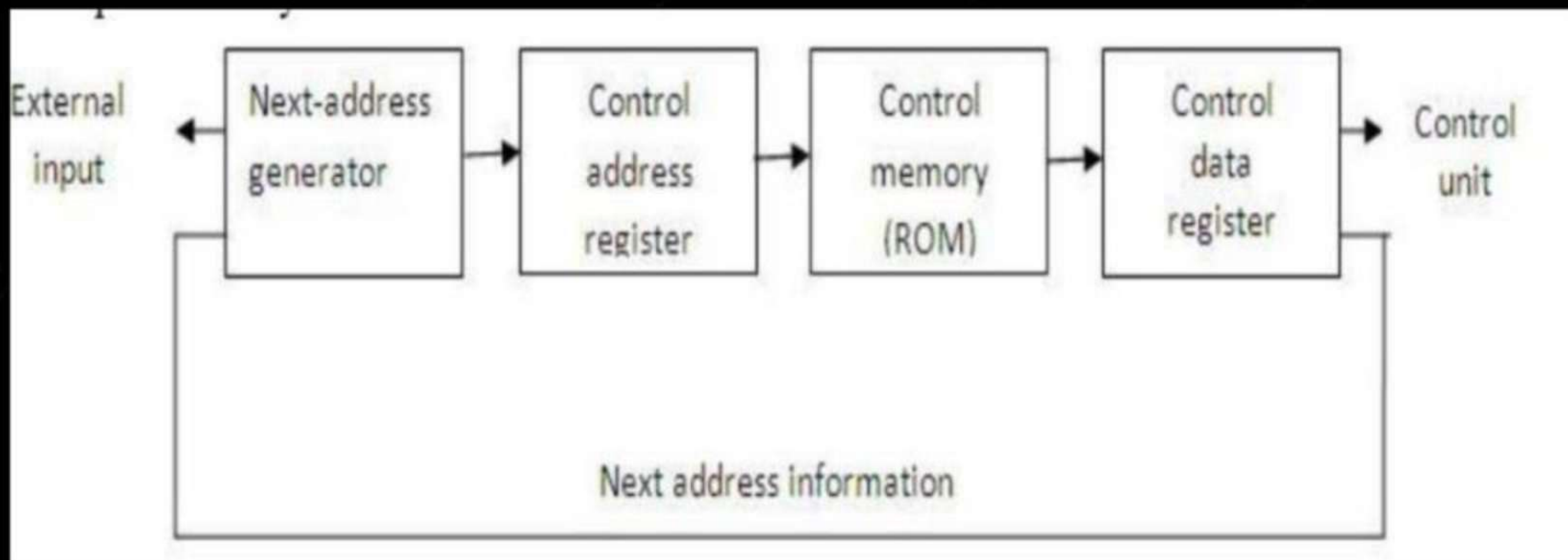
A microprogram is a **sequence of microinstructions** stored in **control memory** that defines **how a CPU executes machine instructions**. It acts as a **firmware layer** between **hardware** and **machine instructions**.

Example: A microprogram for an ADD instruction contains a series of microinstructions to fetch operands, perform addition, and store results.

Microcode

- Microcode is the **binary representation** of a microprogram stored **in control memory**. It consists of microinstructions that control the CPU's internal components.
- Example: A CPU's microcode tells it how to execute complex instructions like multiplication or division internally.

➤ BLOCK DAIGRAM OF MICRO PROGRAMMMED CONTROL UN IT



The **control memory** is assumed to be a **ROM**, within **which all control information** is **permanently stored**.

- The control memory address **register(CAR)** specifies the **address of the microinstruction**, and the control data register (CDR) holds the **microinstruction read from memory**.
- The **microinstruction** contains a **control word** that specifies one **or more micro-operations for the data processor**. Once these operations are executed, the control must determine the next address.

- The location of the **next microinstruction** may be the **one next in sequence**, or it may be located somewhere else in the **control memory**.
- While the **micro-operations** are **being executed**, the **next address** is computed in the **next address generator** circuit and then transferred into the **control address register** to read **the next microinstruction**.
- Thus a **microinstruction** contains **bits for initiating micro-operations** in the **data processor** part and bits that **determine the address sequence** for the control memory.

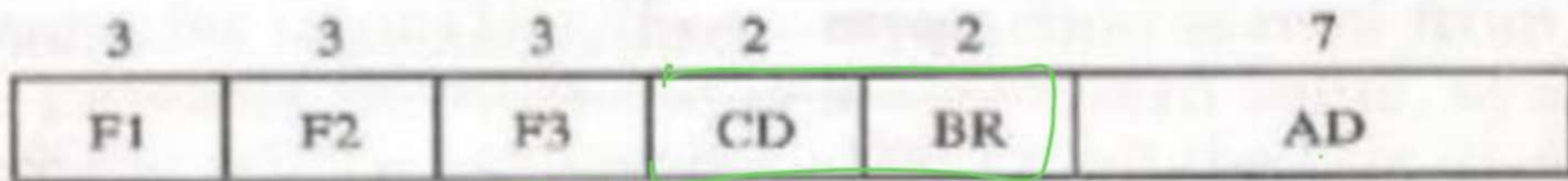
The **next address generator** is sometimes **called a microprogram sequencer**. It is used to generate the next micro instruction address.

Microinstruction Format

➤ The microinstruction format for the control memory is shown in figure The 20 bits of the microinstruction are divided into four functional parts as follows:

1. The three fields **F1, F2, and F3 specify microoperations for the computer.** The microoperations are subdivided **into three fields of three bits each.** The three bits in each field are encoded to specify seven distinct micro-operations. This gives a total of 21 micro-operations.
2. The CD field selects status bit conditions.
3. The BR field specifies the type of branch to be used.
4. The AD field contains a branch address. The address field is seven bits wide,

Microinstruction Format



F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

Microinstruction Format

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0-10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

Microinstruction Format

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0-10) \leftarrow PC$	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow \overline{AC}$	COM
011	$AC \leftarrow \text{shl } AC$	SHL
100	$AC \leftarrow \text{shr } AC$	SHR
101	$PC \leftarrow PC + 1$	INCPC
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
<u>01</u>	<u>DR(15)</u>	<u>I</u>	Indirect address bit
10	AC(15)	S	Sign bit of AC
11	AC = 0	Z	Zero value in AC

BR	Symbol	Function
00	JMP	$\underline{CAR} \leftarrow \underline{AD}$ if condition = 1 $\underline{CAR} \leftarrow \underline{CAR} + 1$ if condition = 0
<u>01</u>	CALL	$\underline{CAR} \leftarrow \underline{AD}$, $\underline{SBR} \leftarrow \underline{CAR} + 1$ if condition = 1 $\underline{CAR} \leftarrow \underline{CAR} + 1$ if condition = 0
10	RET	$\underline{CAR} \leftarrow \underline{SBR}$ (Return from subroutine)
11	MAP	$\underline{CAR}(2-5) \leftarrow \underline{DR}(11-14)$, $\underline{CAR}(0,1,6) \leftarrow 0$

Hardwired Control Unit

Hardwired control unit generates the **control signals** needed for the **processor** using logic **circuits**

Hardwired control unit **is faster** when compared to **microprogrammed control unit** as the required **control signals** are generated with the help of **hardware**

Difficult to modify as the **control signals** that need to be generated are **hard wired**

Microprogrammed Control Unit

Microprogrammed control unit generates the **control signals** with the help of **micro instructions** stored in **control memory**

This is **slower** than the other as **micro instructions** are used for **generating signals** here

Easy to modify as the **modification** need to be done only at the **instruction level**

More costlier as everything has to be realized in terms of logic gates

It cannot handle complex instructions as the circuit design for it becomes complex

Only limited number of instructions are used due to the hardware implementation

Used in computer that makes use of Reduced Instruction Set Computers(RISC)

It can use limited instructions

Less costlier than hardwired control as only micro instructions are used for generating control signals

It can handle complex instructions

Control signals for many instructions can be generated

Used in computer that makes use of Complex Instruction Set Computers(CISC)

It can generate control signal for many instructions

TYPES OF MICROPROGRAMMED CONTROL UNIT

➤ The micro-programmed control unit can be classified into two types based on the type of Control Word stored in the Control Memory, viz., Horizontal micro-programmed control unit and Vertical micro-programmed control unit.



15 Control Signals

➤ In the Horizontal micro-programmed control unit, the control signals are represented in the decoded binary format, i.e., 1 bit/CS. Here 'n' control signals require n bit encoding.

➤ In a Vertical micro-programmed control unit, the control signals are represented in the encoded binary format. Here 'n' control signals require $\log_2 n$ bit encoding.

S. No	Horizontal μ -programmed CU	Vertical μ -programmed CU
1.	It supports <u>longer control word</u> .	It supports <u>shorter control word</u> .
2.	It allows a <u>higher degree of parallelism</u> . If <u>degree is n</u> , then <u>n Control Signals</u> are enabled at a time.	It allows a low degree of parallelism i.e., the degree of parallelism is <u>either 0 or 1</u> .
3.	<u>No additional hardware</u> is required.	<u>Additional hardware</u> in the form of <u>decoders</u> is required to generate control signals.
4.	It is <u>faster</u> than a <u>Vertical micro-programmed</u> control unit.	it is <u>slower than</u> a <u>Horizontal micro-programmed</u> control unit.

Horizontal μ -programmed CU

Vertical μ -programmed CU

5.

It is more flexible than a vertical micro-programmed control unit.

It is less flexible than horizontal but more flexible than that of a hardwired control unit.

6.

A horizontal micro-programmed control unit uses horizontal micro-instruction, where every bit in the control field attaches to a control line.

A vertical micro-programmed control unit uses vertical micro-instruction, where a code is used for each action to be performed and the decoder translates this code into individual control signals.

NUMERICALS ON MICROPROGRAMMED CONTROL UNIT

Q1 Consider a micro programmed control unit design which support 256 instructions each instruction consumes 16 micro operations on an average. The system is using 48 control signals, 16 flags must be supported for taking branching decision. What is the size of control memory (in bytes) for horizontal and vertical micro programmed control unit design respectively?

total instruction = 256
each instruction = 16 uoperation
consume
total microoperation = $256 * 16$
 $2^8 * 2^4 = 2^{12}$ uop.

2⁰ → 2
2¹ → 4
2² → 8
2³ → 16
2⁴ → 32
2⁵ → 64
2⁶ → 128
2⁷ → 256
2⁸ → 512
2⁹ → 1024
2¹⁰ → 2048
2¹¹ → 4096
2¹² → 8192
2¹³ → 16384
2¹⁴ → 32768
2¹⁵ → 65536
2¹⁶ → 131072
2¹⁷ → 262144
2¹⁸ → 524288
2¹⁹ → 1048576
2²⁰ → 2097152
2²¹ → 4194304
2²² → 8388608
2²³ → 16777216
2²⁴ → 33554432
2²⁵ → 67108864
2²⁶ → 134217728
2²⁷ → 268435456
2²⁸ → 536870912
2²⁹ → 1073741824
2³⁰ → 2147483648
2³¹ → 4294967296
2³² → 8589934592
2³³ → 17179869184
2³⁴ → 34359738368
2³⁵ → 68719476736
2³⁶ → 137438953472
2³⁷ → 274877906944
2³⁸ → 549755813888
2³⁹ → 1099511627776
2⁴⁰ → 2199023255552
2⁴¹ → 4398046511104
2⁴² → 8796093022208
2⁴³ → 17592186044416
2⁴⁴ → 35184372088832
2⁴⁵ → 70368744177664
2⁴⁶ → 140737488355328
2⁴⁷ → 281474976710656
2⁴⁸ → 562949953421312
2⁴⁹ → 1125899906842624
2⁵⁰ → 2251799813685248
2⁵¹ → 4503599627370496
2⁵² → 9007199254740992
2⁵³ → 18014398509481984
2⁵⁴ → 36028797018963968
2⁵⁵ → 72057594037927936
2⁵⁶ → 144115188075855872
2⁵⁷ → 288230376151711744
2⁵⁸ → 576460752303423488
2⁵⁹ → 1152921504606846976
2⁶⁰ → 2305843009213693952
2⁶¹ → 4611686018427387904
2⁶² → 9223372036854775808
2⁶³ → 18446744073709551616
2⁶⁴ → 36893488147419103232
2⁶⁵ → 73786976294838206464
2⁶⁶ → 147573952589676412928
2⁶⁷ → 295147905179352825856
2⁶⁸ → 590295810358705651712
2⁶⁹ → 1180591620717411303424
2⁷⁰ → 2361183241434822606848
2⁷¹ → 4722366482869645213696
2⁷² → 9444732965739290427392
2⁷³ → 18889465931478580854784
2⁷⁴ → 37778931862957161709568
2⁷⁵ → 75557863725914323419136
2⁷⁶ → 151115727451828646838272
2⁷⁷ → 302231454903657293676544
2⁷⁸ → 604462909807314587353088
2⁷⁹ → 1208925819614629174706176
2⁸⁰ → 2417851639229258349412352
2⁸¹ → 4835703278458516698824704
2⁸² → 9671406556917033397649408
2⁸³ → 19342813113834066795298816
2⁸⁴ → 38685626227668133590597632
2⁸⁵ → 77371252455336267181195264
2⁸⁶ → 154742504910672534362390528
2⁸⁷ → 309485009821345068724781056
2⁸⁸ → 618970019642690137449562112
2⁸⁹ → 1237940039285380274899124224
2⁹⁰ → 2475880078570760549798248448
2⁹¹ → 4951760157141521099596496896
2⁹² → 9903520314283042199192993792
2⁹³ → 19807040628566084398385987584
2⁹⁴ → 39614081257132168796771975168
2⁹⁵ → 79228162514264337593543950336
2⁹⁶ → 158456325028528675187087900672
2⁹⁷ → 316912650057057350374175801344
2⁹⁸ → 633825300114114700748351602688
2⁹⁹ → 1267650600228229401496703205376
2¹⁰⁰ → 2535301200456458802993406410752
2¹⁰¹ → 5070602400912917605986812821504
2¹⁰² → 10141204801825835211973625643008
2¹⁰³ → 20282409603651670423947251286016
2¹⁰⁴ → 40564819207303340847894502572032
2¹⁰⁵ → 81129638414606681695789005144064
2¹⁰⁶ → 162259276829213363391578010288128
2¹⁰⁷ → 324518553658426726783156020576256
2¹⁰⁸ → 649037107316853453566312041152512
2¹⁰⁹ → 1298074214633706907132624082305024
2¹¹⁰ → 2596148429267413814265248164610048
2¹¹¹ → 5192296858534827628530496329220096
2¹¹² → 10384593717069655257060992658440192
2¹¹³ → 20769187434139310514121985316880384
2¹¹⁴ → 41538374868278621028243970633760768
2¹¹⁵ → 83076749736557242056487941267521536
2¹¹⁶ → 166153499473114484112975882535043072
2¹¹⁷ → 332306998946228968225951765070086144
2¹¹⁸ → 664613997892457936451903530140172288
2¹¹⁹ → 1329227995784915872903807060280344576
2¹²⁰ → 2658455991569831745807614120560689152
2¹²¹ → 5316911983139663491615228241121378304
2¹²² → 10633823966279326983230456482242756608
2¹²³ → 21267647932558653966460912964485513216
2¹²⁴ → 42535295865117307932921825928971026432
2¹²⁵ → 85070591730234615865843651857942052864
2¹²⁶ → 170141183460469231731687303715884105728
2¹²⁷ → 340282366920938463463374607431768211456
2¹²⁸ → 680564733841876926926749214863536422912
2¹²⁹ → 1361129467683753853853498429727072845824
2¹³⁰ → 2722258935367507707706996859454145691648
2¹³¹ → 5444517870735015415413993718908291383296
2¹³² → 10889035741470030830827987437816582766592
2¹³³ → 21778071482940061661655974875633165533184
2¹³⁴ → 43556142965880123323311949751266331066368
2¹³⁵ → 87112285931760246646623899502532662132736
2¹³⁶ → 174224571863520493293247799005065324265472
2¹³⁷ → 348449143727040986586495598010130648530944
2¹³⁸ → 696898287454081973172991196020261297061888
2¹³⁹ → 1393796574908163946345982392040522594123776
2¹⁴⁰ → 2787593149816327892691964784081045188247552
2¹⁴¹ → 5575186299632655785383929568162090376495104
2¹⁴² → 11150372599265311570767859136324180752990208
2¹⁴³ → 22300745198530623141535718272648361505980416
2¹⁴⁴ → 44601490397061246283071436545296723011960832
2¹⁴⁵ → 89202980794122492566142873090593446023921664
2¹⁴⁶ → 178405961588244985132285746181186892047843328
2¹⁴⁷ → 356811923176489970264571492362373784095686656
2¹⁴⁸ → 713623846352979940529142984724747568191373312
2¹⁴⁹ → 1427247692705959881058285969449495136382746624
2¹⁵⁰ → 2854495385411919762116571938898990272765493248
2¹⁵¹ → 5708990770823839524233143877797980545530986496
2¹⁵² → 11417981541647679048466287755595961091061972992
2¹⁵³ → 22835963083295358096932575511191922182123945984
2¹⁵⁴ → 45671926166590716193865151022383844364247891968
2¹⁵⁵ → 91343852333181432387730302044767688728495783936
2¹⁵⁶ → 182687704666362864775460604089535377456991567872
2¹⁵⁷ → 365375409332725729550921208179070754913983135744
2¹⁵⁸ → 730750818665451459101842416358141509827966271488
2¹⁵⁹ → 1461501637330902918203684832716283019655932542976
2¹⁶⁰ → 2923003274661805836407369665432566039311865085952
2¹⁶¹ → 5846006549323611672814739330865132078623730171904
2¹⁶² → 11692013098647223345629478661730264157247460343808
2¹⁶³ → 23384026197294446691258957323460528314494920687616
2¹⁶⁴ → 46768052394588893382517914646921056628989841375232
2¹⁶⁵ → 93536104789177786765035829293842113257979682750464
2¹⁶⁶ → 187072209578355573530071658587684226515959365500928
2¹⁶⁷ → 374144419156711147060143317175368453031918731001856
2¹⁶⁸ → 748288838313422294120286634350736906063837462003712
2¹⁶⁹ → 1496577676626844588240573268701473812127674924007424
2¹⁷⁰ → 2993155353253689176481146537402947624255349848014848
2¹⁷¹ → 5986310706507378352962293074805895248510699696029696
2¹⁷² → 11972621413014756705924586149611790497021399392059392
2¹⁷³ → 23945242826029513411849172299223580994042798784118784
2¹⁷⁴ → 47890485652059026823698344598447161988085597568237568
2¹⁷⁵ → 95780971304118053647396689196894323976171195136475136
2¹⁷⁶ → 191561942608236107294793378393788647952342390272950272
2¹⁷⁷ → 383123885216472214589586756787577295904684780545900544
2¹⁷⁸ → 766247770432944429179173513575154591809369561091801088
2¹⁷⁹ → 1532495540865888858358347027150309183618739122183602176
2¹⁸⁰ → 3064991081731777716716694054300618367237478244367204352
2¹⁸¹ → 6129982163463555433433388108601236734474956488734408704
2¹⁸² → 12259964326927110866866776217202473468949912977468817408
2¹⁸³ → 24519928653854221733733552434404946937899825954937634816
2¹⁸⁴ → 49039857307708443467467104868809893875799651909875269632
2¹⁸⁵ → 98079714615416886934934209737619787751599303819750539264
2¹⁸⁶ → 196159429230833773869868419475239575503198607639501078528
2¹⁸⁷ → 392318858461667547739736838950479151006397215279002157056
2¹⁸⁸ → 784637716923335095479473677900958302012794430558004314112
2¹⁸⁹ → 1569275433846670190958947355801916604025588861116008628224
2¹⁹⁰ → 3138550867693340381917894711603833208051177722232017256448
2¹⁹¹ → 6277101735386680763835789423207666416102355444464034512896
2¹⁹² → 12554203470773361527671578846415332832204710888928069025792
2¹⁹³ → 25108406941546723055343157692830665664409421777856138051584
2¹⁹⁴ → 50216813883093446110686315385661331328818843555712276103168
2¹⁹⁵ → 100433627766186892221372630771322662657637687111424552206336
2¹⁹⁶ → 200867255532373784442745261542645325315275374222849104412672
2¹⁹⁷ → 401734511064747568885490523085290650630550748445698208825344
2¹⁹⁸ → 803469022129495137770981046170581301261101496891396417650688
2¹⁹⁹ → 1606938044258990275541962092341162602522202993782792835301376
2²⁰⁰ → 3213876088517980551083924184682325205044405987565585670602752
2²⁰¹ → 6427752177035961102167848369364650410088811975131171341205504
2²⁰² → 12855504354071922204335696738729300820177623950262342682411008
2²⁰³ → 25711008708143844408671393477458601640355247900524685364822016
2²⁰⁴ → 51422017416287688817342786954917203280710495801049370729644032
2²⁰⁵ → 102844034832575377634685573909834406561420991602098741459288064
2²⁰⁶ → 205688069665150755269371147819668813122841983204197482918576128
2²⁰⁷ → 411376139330301510538742295639337626245683966408394965837152256
2²⁰⁸ → 822752278660603021077484591278675252491367932816789931674304512
2²⁰⁹ → 1645504557321206042154969182557350504982735865633579863348609024
2²¹⁰ → 3291009114642412084309938365114701009965471731267159726697218048
2²¹¹ → 6582018229284824168619876730229402019930943462534319453394436096
2²¹² → 13164036458569648337239753460458804039861886925068638906788872192
2²¹³ → 26328072917139296674479506920917608079723773850137277813577744384
2²¹⁴ → 52656145834278593348959013841835216159447547700274555627155488768
2²¹⁵ → 105312291668557186697918027683670432318895095400549111254310977536
2²¹⁶ → 210624583337114373395836055367340864637790190801098222508621955072
2²¹⁷ → 421249166674228746791672110734681729275580381602196445017243910144
2²¹⁸ → 842498333348457493583344221469363458551160763204392890034487820288
2²¹⁹ → 1684996666696914987166688442938726917102321526408785780068975640576
2²²⁰ → 3369993333393829974333376885877453834204643052817571560137951281152
2²²¹ → 6739986666787659948666753771754907668409286105635143120275902562304
2²²² → 13479973333575319897333507543509815336818572211270286240551805124608
2²²³ → 26959946667150639794667015087019630673637144422540572481103610249216
2²²⁴ → 53919893334301279589334030174039261347274288845081144962207220498432
2²²⁵ → 107839786668602559178668060348078522694548577690162289924414440996864
2²²⁶ → 215679573337205118357336120696157045389097155380324579848828881993728
2²²⁷ → 431359146674410236714672241392314090778194310760649159697657763987456
2²²⁸ → 862718293348820473429344482784628181556388621521298319395315527974912
2²²⁹ → 1725436586697640946858688965569256363112777243042596638790631055949824
2²³⁰ → 3450873173395281893717377931138512726225554486085193277581262111899648
2²³¹ → 6901746346790563787434755862277025452451108972170386555162524223799296
2²³² → 13803492693581127574869511724554050904902217944340773110325048447598592
2²³³ → 27606985387162255149739023449108101809804435888681546220650096895197184
2²³⁴ → 55213970774324510299478046898216203619608871777363092441300193790394368
2²³⁵ → 110427941548649020598956093796432407239217743554726184882600387580788736
2²³⁶ → 220855883097298041197912187592864814478435487109452369765200775161577472
2²³⁷ → 441711766194596082395824375185729628956870974218904739530401550323154944
2²³⁸ → 8834235323891921647916487503714592579137419484378094790608

Horizontal

One u-Instruction^{op} = $48 + 4 + \frac{12}{8} = 64 \text{ bit}$

$64 * 2^{12} \text{ bit}$

total bit needed
for 2^{12} uoperation = $\frac{64 * 12}{8}$

$$2^3 \times 2^{12} = 2^{15} \text{ Byte}$$

$$2^5 * 2^{10} \text{ B}$$

$$2^5 \text{ KB}$$

$$\underline{32 \text{ KB}}$$

— Size of Control Memory

Vertical Microprogrammed

Size of microinstruction: $6+4+12 = 22$ bit

micro op

2^{12} Microinstruction = $2^{12} * 22$ bit

need bit

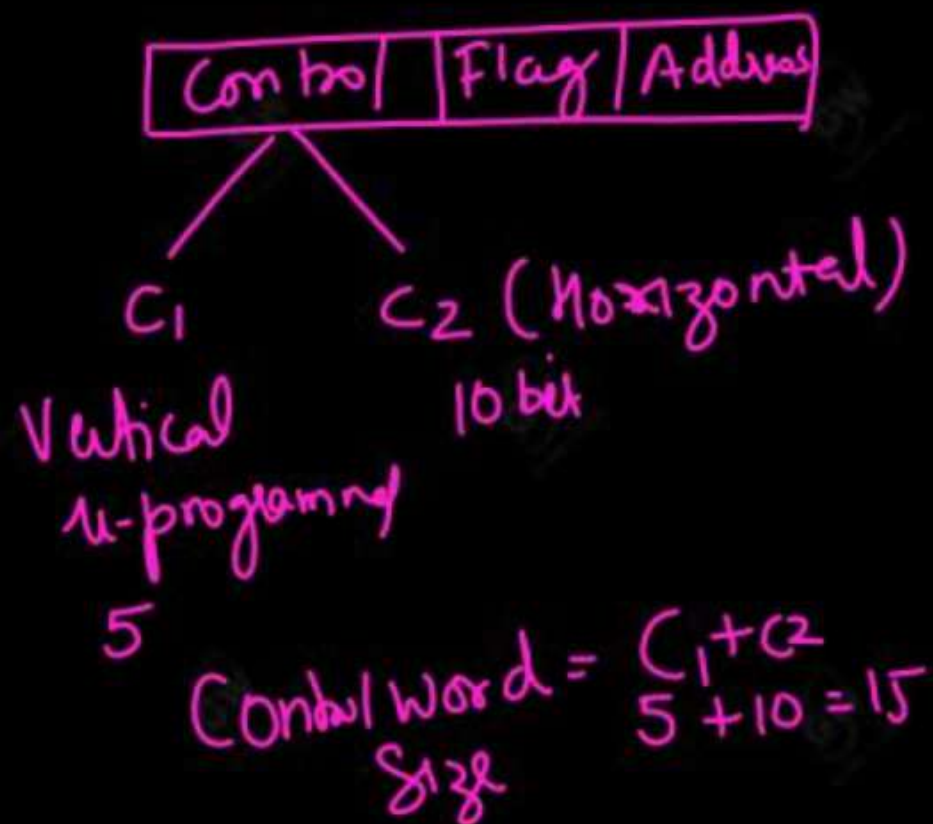
$$\frac{2^{12} * 22}{8} \text{ byte} = \frac{2^{12} * 2 * 11}{2^3}$$

$$= \frac{2^{10} * 2^3 * 11}{2^3} = 11 \text{ KB}$$

NUMERICALS ON MICROPROGRAMMED CONTROL UNIT

Q3 consider a hypothetical control unit that supports 2 groups of control signal group 1 indicate none or one 30 control signal group 2 indicate 10 almost control signal what is size of control word or signal

AKTU



2^0	— 1
2^1	— 2
2^2	— 4
2^3	— 8
2^4	— 16
2^5	— 32
2^6	— 64

NUMERICALS ON MICROPROGRAMMED CONTROL UNIT

Q4 A vertical micro programmed control unit support 512 instructions . The system is using 8 conditional flag and contain 31 control signal .Each instruction on an average has 1 micro operation .calculate the approximate size of memory in bytes (AKTU 2021-22)

(Vertical)

No of instruction = 512

Each instruction = 1 microoperation

8 conditional flag

31 control signal

No of bit need to Represent

8 conditional flag $= 2^3 = 3 \text{ bit}$

total microoperation = $512 * 1 = 512 = 2^9 \text{ microoperation}$
No of bit needed to Represent 512 uop
 $2^9 = 9 \text{ bits}$

31 control signal needed bit = 5 bits

$2^0 \rightarrow 1$
 $2^1 \rightarrow 2$
 $2^2 \rightarrow 4$
 $2^3 \rightarrow 8$
 $2^4 \rightarrow 16$
 $2^5 \rightarrow 32$
 $2^6 \rightarrow 64$



No of bits to represent one μ -instruction = $5 + 3 + 9 = 17$ bits
 No of bits to represent 2^9 operations = $2^9 * 17$ bits

$$\frac{2^9 * 17}{8} \text{ byte}$$

$$\frac{2^9 * 17}{2^3} = 2^6 * 17$$

$$64 * 17$$

1088 Byte - Size of Control Memory

Numerical on pipeline

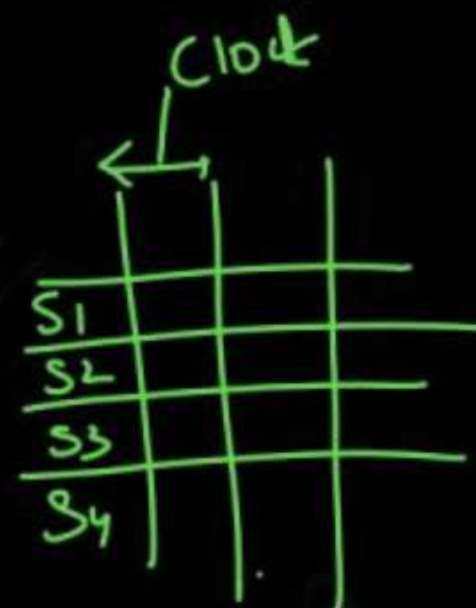
AKTU

Problem-01:

Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-

- Pipeline cycle time
- Non-pipeline execution time
- Speed up ratio
- Pipeline time for 1000 tasks
- Sequential time for 1000 tasks
- Throughput

Pipeline cycle time
= 1 clock time
Max of any stage + Register or
latch delay
 $\text{Max}\{60, 50, 90, 80\} + 10$
 $90 \text{ ns} + 10 \text{ ns} = 100 \text{ ns}$
Pipeline cycle = 100 ns



(ii) Non-pipeline execution time for 1 instruction
 = time to go through all stages
 $60 + 60 + 90 + 80 = 280 \text{ ns}$

(iii)
$$\text{Speedup} = \frac{\text{Non-pipeline Execution time}}{\text{Pipeline Execution time}}$$

$$\frac{280 \text{ ns}}{100 \text{ ns}} = 2.8$$

efficiency = $\frac{\text{Speedup}}{\text{No of stage in pipelined}} = \frac{2.8}{4} = 0.7$

(iv) pipeline time for 1000 task
 first ins + Rem
 $K * \text{cycle time} + (n-1) * \text{cycle time}$
 $(K + n-1) * \text{cycle time}$
 $(4 + 1000 - 1) * 100$
 $1003 * 100 = 100300$

Non-pipelined

$$\begin{aligned}\text{Execution time} &= \text{One instruction Execution time} * \text{No of instructions} \\ &280 * 1000 \\ &280000 \text{ ns}\end{aligned}$$

throughput
(pipeline)

$$\begin{aligned}&= \frac{\text{No of ins}^n}{\text{total time}} \\ &\quad \text{in. 1000 ins} \\ &\frac{1000}{100300}\end{aligned}$$

Problem -2

A four stage pipeline has the stage delays as 150, 120, 160 and 140 ns respectively. Registers are used between the stages and have a delay of 5 ns each. Assuming constant clocking rate, the total time taken to process 1000 data items on the pipeline will be

Answer will be in micro second

$$\begin{aligned} \text{Cycletime} &= \text{Max of any Stage} + \text{Register delay} \\ \text{ON pipeline} & \quad \text{Max} \{150, 120, 160, 140\} + 5 \\ \text{Execution time} & \quad 160 + 5 = 165 \text{ ns} \end{aligned}$$

total time taken to process 1000 ns

$$(K+n-1) * \text{cycle time}$$

$$(4+1000-1) * 165$$

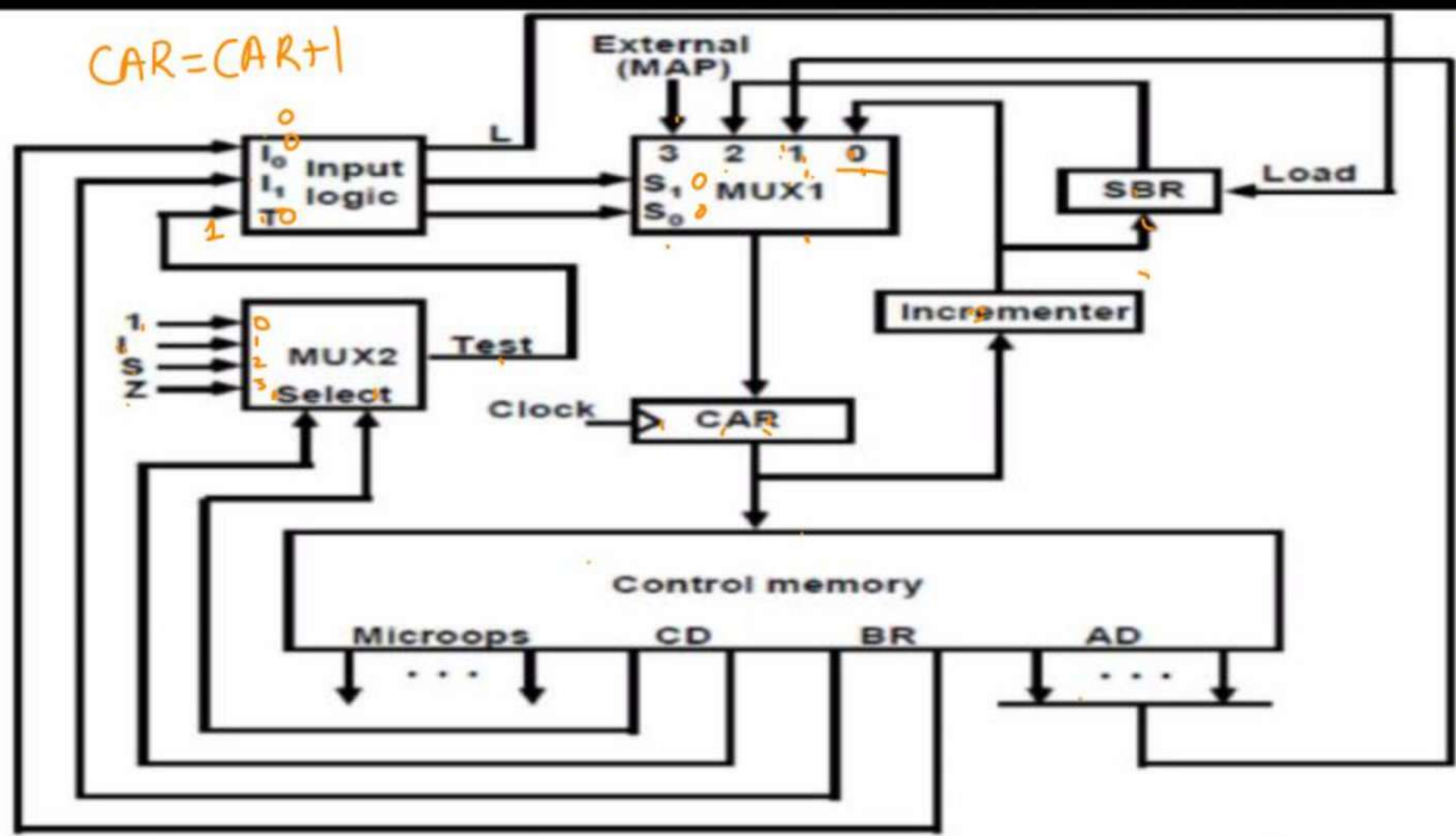
$$1003 * 165$$

$$165495 \text{ ns}$$

$$165.495 * 10^{-3} \text{ ns}$$

$$165.495 \mu\text{s}$$

MICROPROGRAM SEQUENCER



CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	$DR(15)$	I	Indirect address bit
10	$AC(15)$	S	Sign bit of AC
11	$AC = 0$	Z	Zero value in AC

MICROPROGRAM SEQUENCER

BR FIELD		INPUT			MUX1		loadSBR
		I_0	I_1	T	S1	S2	/
0	0	0	0	0	0	0	0
	0	0	0	1	0	1	0
0	1	0	1	0	0	0	0
	1	0	1	1	0	1	1
1	0	1	0	X	1	0	0
1	1	1	1	X	1	1	0

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD, SBR \leftarrow CAR + 1$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

MICROPROGRAM SEQUENCER

The basic components of a microprogrammed control unit are the control memory and the circuits that select the next address.

- The address selection part is called a microprogram sequencer.
- The purpose of a microprogram sequencer is to present an address to the control memory so that a microinstruction may be read and executed.
- The next-address logic of the sequencer determines the specific address source to be loaded into the control address register.
- The block diagram of the microprogram sequencer is shown in
- The control memory is included in the diagram to show the interaction between the sequencer and the memory attached to it.

MICROPROGRAM SEQUENCER

- There are two multiplexers in the circuit.
- The first multiplexer selects an address from one of four sources and routes it into control address register *CAR*. The second multiplexer tests the value of a selected status bit and the result of the test is applied to an input logic circuit.
- The output from *CAR* provides the address for the control memory. The content of *CAR* is incremented and applied to one of the multiplexer inputs and to the subroutine register *SBR*.
- The other three inputs to multiplexer come from
- The address field of the present microinstruction
- From the out of *SBR*
- From an external source that maps the instruction

MICROPROGRAM SEQUENCER

- The CD (condition) field of the microinstruction selects one of the status bits in the second multiplexer.
- If the bit selected is equal to 1, the T variable is equal to 1; otherwise, it is equal to 0.
- The *T* value together with two bits from the BR (branch) field goes to an input logic circuit.
- The input logic in a particular sequencer will determine t
- The input logic circuit in above figure has three inputs I0, I1, and T, and three outputs, S0, S1, and L.
- Variables S0 and S1 select one of the source addresses for CAR. Variable L enables the load input in SBR.
- The binary values of the selection variables determine the path in the multiplexer.
- For example, with $S1, S0 = 10$, multiplexer input number 2 is selected and establishes transfer path from SBR to CAR.
- The truth table for the input logic circuit is shown in Table b.
- The type of operations that are available in the unit.

➤ Instruction formats refer to the way instructions are encoded and represented in machine language.

There are several types of instruction formats, including zero, one, two, and three-address instructions

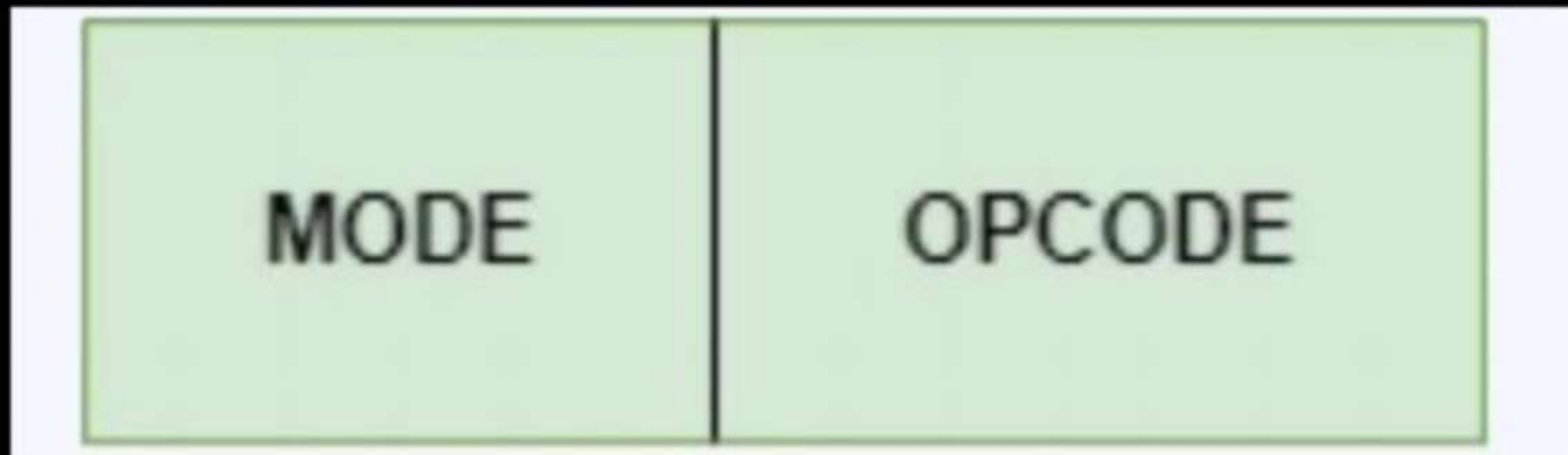
➤ An opcode is a collection of bits that represents the basic operations including add, subtract, multiply, complement, and shift.

➤ Operands are definite elements of computer instruction that show what information is to be operated on

$a+b$

Zero address instruction

This instruction does not have an operand field, and the location of operands is implicitly represented. The stack-organized computer system supports these instructions. To evaluate the arithmetic expression, it is required to convert it into reverse polish notation.



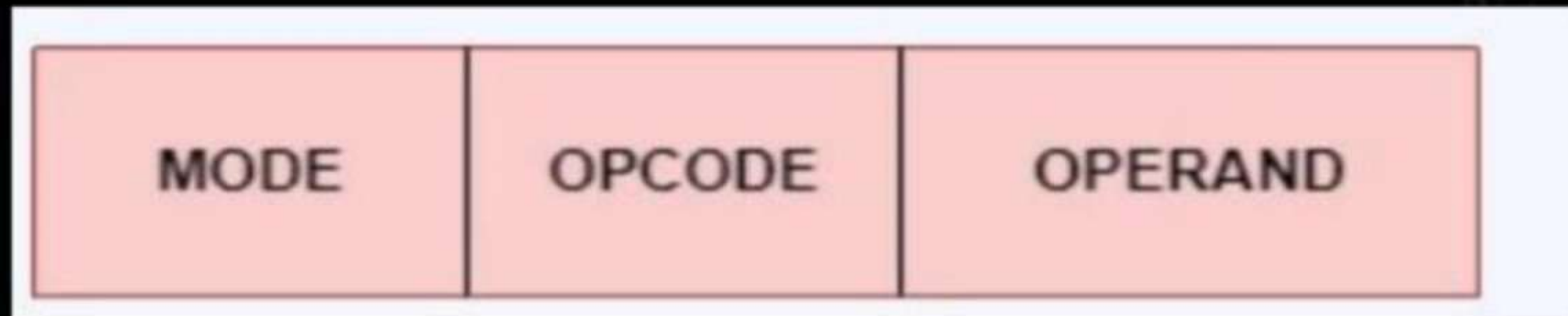
Zero address instruction

TOS: Top of the Stack Expression: $X = (A+B)*(C+D)$ Post fix : $X = AB+CD+*$

PUSH A	TOS $\leftarrow$ A
PUSH B	TOS $\leftarrow$ B
ADD	TOS $\leftarrow$ (A + B)
PUSH C	TOS $\leftarrow$ C
PUSH D	TOS $\leftarrow$ D
ADD	TOS $\leftarrow$ (C + D)
MUL	TOS $\leftarrow$ (C + D) * (A + B)
POP X	M [X] $\leftarrow$ TOS

} PUSH
PUSH
ADD
PUSH
PUSH

- This instruction uses an implied accumulator for data manipulation operations.
- An accumulator is a register used by the CPU to perform logical operations.
- In one address instruction, the accumulator is implied, and hence, it does not require an explicit reference.
- For multiplication and division, there is a need for a second register. However, here we will neglect the second register and assume that the accumulator contains the result of all the operations.



➤ **Example:** The program to evaluate $X = (A + B) * (C + D)$ is as follows:

➤ LOAD A

➤ ADD B

➤ STORE T

➤ LOAD C

➤ ADD D

➤ MUL T

➤ STORE X

AC $\leftarrow$ M [A]

AC $\leftarrow$ AC + M [B]

M [T] $\leftarrow$ AC

AC $\leftarrow$ M [C]

AC $\leftarrow$ AC + M [D]

AC $\leftarrow$ AC * M [T]

M [X] $\leftarrow$ AC

45	X
5	T
2	A
3	B
4	C
5	D

- All operations are done between the accumulator(AC) register and a memory operand.
- $M[]$ is any memory location.
- $M[T]$ addresses a temporary memory location for storing the intermediate result.
- This instruction format has only one operand field. This address field uses two special instructions to perform data transfer,
 - namely:
 - **LOAD:** This is used to transfer the data to the accumulator.
 - **STORE:** This is used to move the data from the accumulator

Two address instruction

- These instructions specify two operand or address, which typically refers to a memory location or register. The instruction operates on the contents of that operand, and the result may be stored in the same or a different location.



Two address instruction

- This is common in commercial computers.
- Here two addresses can be specified in the instruction. Unlike earlier in one address instruction, the result was stored in the accumulator, here the result can be stored at different locations rather than just accumulators, but require more number of bit to represent address.

Mode	opcode	Destination address	Source address
------	--------	---------------------	----------------

Two address instruction

Example: The program to evaluate $X = \overset{R_1}{(A + B)} * \overset{R_2}{(C + D)}$ is as follows:

✓ MOV R1, A

$R1 \leftarrow M[A]$

✓ ADD R1, B

$R1 \leftarrow R1 + M[B]$

MOV R2, C

$R2 \leftarrow M[C]$

ADD R2, D

$R2 \leftarrow R2 + M[D]$

MUL R1, R2

$R1 \leftarrow R1 * R2$

MOV X, R1

$M[X] \leftarrow R1$

$R_1 \leftarrow C$

The MOV instruction transfers the operands to the memory from the processor registers. R1, R2 registers.

Three address instruction

These instructions specify three operands or addresses, which may be memory locations or registers. The instruction operates on the contents of all three operands, and the result may be stored in the same or a different location



Three address instruction

- This has three address field to specify a register or a memory location.
- Program created are much short in size but number of bits per instruction increase.
- These instructions make creation of program much easier but it does not mean that program will run much faster because now instruction only contain more information but each micro operation (changing content of register, loading address in address bus etc.) will be performed in one cycle only

Mode	opcode	Destination address	Source address	Source address

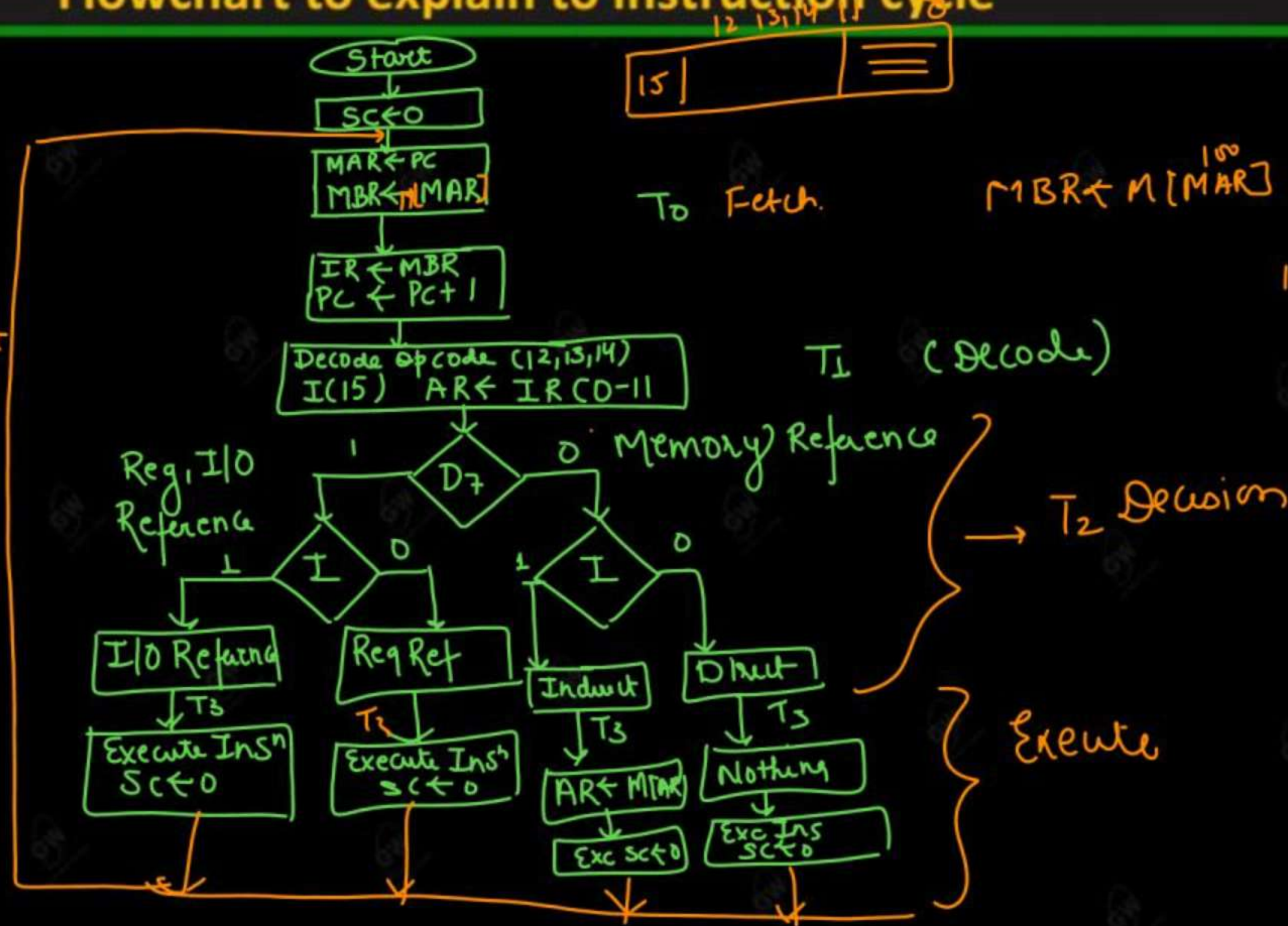
Three address instruction

$$X = (A+B) * (C*D)$$

ADD <u>R1</u> , A, B	<u>$R1 = M[A] + M[B]$</u>
ADD <u>R2</u> , C, D	↓ <u>$R2 = M[C] + M[D]$</u>
MUL <u>X</u> , R1, R2	<u>$M[X] = R1 * R2$</u>

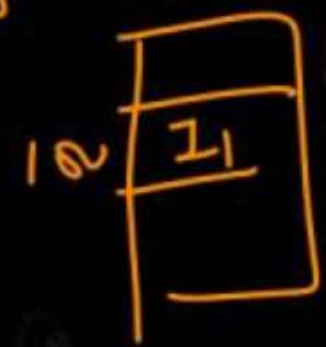
Flowchart to explain to instruction cycle

MAR ← Memory Address Reg.
Insⁿ → Instruction
I/O - Input Output
Reference
Reg - Register.



To Fetch.

$MBR \leftarrow M[MAR]$

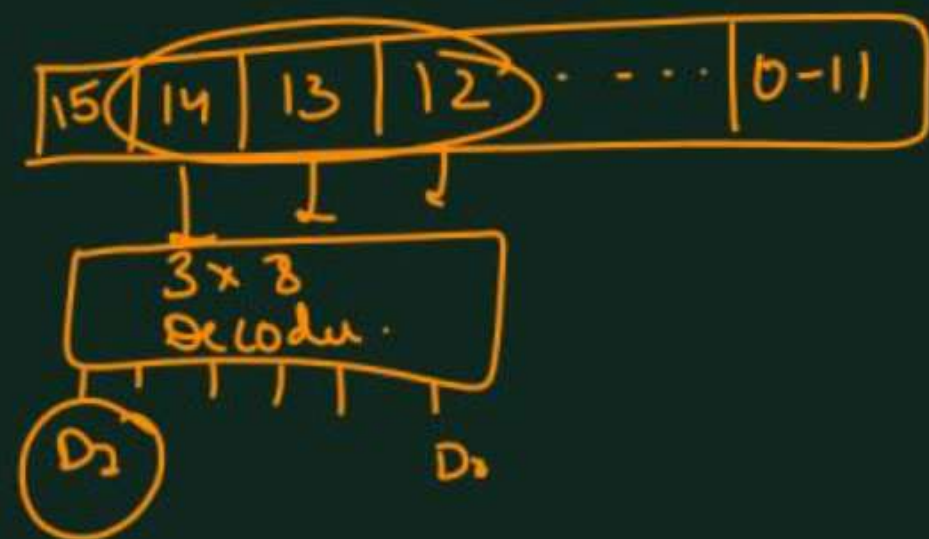


T₁ (Decode)

→ T₂ Decision

Execute

Instruction Reg





Gateway Classes

Empowering Learners, Transforming Futures



Thank You



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53



LIKE

SUBSCRIBE



SHARE

